

BABEȘ-BOLYAI UNIVERSITY CLUJ-NAPOCA
FACULTY OF MATHEMATICS AND COMPUTER
SCIENCE
SPECIALIZATION INFORMATICS IN GERMAN

DIPLOMA THESIS

Chess Engine: From Bitboard Optimizations to Machine-Learned Position Evaluation

Supervisor
Assoc. Prof. Dr. Christian Săcărea

Author
Mândru Cătălin

2025

BABEȘ-BOLYAI UNIVERSITÄT CLUJ-NAPOCA
FAKULTÄT FÜR MATHEMATIK UND INFORMATIK
INFORMATIK IN DEUTSCHER SPRACHE

BACHELORARBEIT

Schach-Engine: Von Bitboard-Optimierungen bis zur Maschinengelernten Positionsevaluierung

Betreuer
Apl. Prof. Dr. Christian Săcărea

Eingereicht von
Mândru Cătălin

2025

UNIVERSITATEA BABEȘ-BOLYAI CLUJ-NAPOCA
FACULTATEA DE MATEMATICĂ ȘI INFORMATICĂ
SPECIALIZAREA Informatică în limba germană

LUCRARE DE LICENȚĂ

**Motor de șah: De la optimizări
Bitboard până la evaluarea poziției cu
învățare automată**

Conducător științific
Conf. Univ. Dr. Christian Săcărea

*Absolvent
Mândru Cătălin*

2025

Abstract

This thesis presents the design, implementation, and evaluation of a chess engine augmented by a learned position-evaluation model. The work proceeds in two main parts. First, the engine’s move-generation and search core are developed from a straightforward object-oriented prototype to a fully bitboard-based, mutable state implementation. Four successive optimization phases: immutable OOP, bitboard caching with magic move tables, in-place move execution with history stacks, and complete migration to bitwise representations; are each benchmarked using six standard perft positions. Through these phases, perft performance improves from 63 k nodes/sec to over 1.6 M nodes/sec, demonstrating the impact of bitwise operations, caching, and minimized object allocation.

Second, a data pipeline and neural-network evaluator are constructed to replace traditional handcrafted evaluation functions. Starting from 200 k positions mined from high-Elo Lichess games (increasing to 5 M); TFRecord serialization enables efficient streaming during training. Neural architectures are studied: a convolutional network that learns local spatial features. The model employs a custom loss function emphasizing balanced positions.

By integrating the learned evaluator into the multi-threaded PVS-based search, this work lays a foundation for engines that combine state-of-the-art performance with data-driven positional understanding. The methodologies and results offer a reproducible path toward modern chess engine development, bridging classical search techniques with contemporary machine-learning approaches.

Contents

1	Introduction	1
1.1	Context and Motivation	1
1.2	Problem Statement and Objectives	2
1.3	Research questions	2
1.4	Scope and Limitations	3
1.5	Contributions	3
2	Background and Literature	5
2.1	Chess as a Computational Problem	5
2.1.1	Rules and Notation	5
2.1.2	Computational Complexity	7
2.1.3	Performance Metrics	8
2.2	Classical Search Algorithms	9
2.2.1	Minimax Algorithm	9
2.2.2	Alpha-Beta Pruning	10
2.2.3	Search Enhancements	11
2.2.4	Board Representations	13
2.2.5	Move-Generation Techniques	14
2.2.6	Material and Positional Factors	16
2.3	Machine-Learning Foundations	16
2.3.1	Machine-Learning Overview	16
2.3.2	Neural Network Foundations for Chess Evaluation	18
2.3.3	Feature Representation	18
2.3.4	Neural Network Architectures	18
2.3.5	Huber Loss Function and Sample Weighting	19
2.3.6	Optimization Algorithms	19
2.3.7	Evaluation Metrics	20
2.4	Benchmarking	20
2.4.1	Perft Testing	20

3	Engine design	22
3.1	Overview and Goals	22
3.2	Project Structure and Tools	22
3.3	Performance and Correctness Tests via Perft	23
3.4	Design Evolution	24
3.4.1	Phase I: OOP and Immutable State	24
3.4.2	Phase II: Move Generation with BitBoards and Caching	27
3.4.3	Phase III: Mutable Core	29
3.4.4	Phase IV: Full BitBoard Migration	30
3.5	Final Engine Flow	32
3.6	Frontend and Backend communication	33
4	Search Algorithm	34
4.1	Search Overview	34
4.2	Alpha-Beta Search	34
4.3	Move Ordering Heuristics	35
4.4	Transposition Tables and Zobrist Hashing	35
4.5	Iterative Deepening	36
4.6	Aspiration Windows	36
4.7	Concurrency Model	36
4.8	Search Statistics	37
4.9	Evaluation Function	38
5	Machine Learning Models	39
5.1	Introduction	39
5.2	TensorFlow Framework	39
5.3	Data Pipeline	40
5.3.1	Data Generation from PGN	40
5.3.2	Data Visualization and Details	40
5.3.3	TFRecord Serialization	41
5.3.4	Feature Encoding: BitBoards vs. One-Hot	42
5.4	Convolutional Neural Network Architecture	42
5.4.1	Initial Baseline (Version 1)	42
5.4.2	Hybrid CNN + Dense Extra-Features (Version 8)	43
5.4.3	Final Convolutional Architecture (Version 9)	45
5.5	Weighted MSE vs. Huber Loss Functions	47
5.6	Training Infrastructure	48
5.7	In-Game Performance	49

6 Conclusion	50
6.1 Summary of Contributions	50
6.2 Answers to Research Questions	50
6.3 Limitations	51
6.4 Future Work	51
Bibliography	53

Chapter 1

Introduction

1.1 Context and Motivation

Since the early 1950s, following Claude Shannon’s paper that formalized the connection between chess and computer programming[Sha50], chess has served as one of the main benchmarks for artificial intelligence research. It is known that chess is *EXPTIME*-complete, meaning that solving for the true outcome of an arbitrary position requires exponential time in the board size or move limit. On a standard 8×8 board, each move branches out into roughly 35 possible legal positions. This is one very large complexity that motivated people to try and create a program that is able to not only play the game at a high level but also master it.

In the year 1997, the victory of IBM’s Deep Blue over the grandmaster Garry Kasparov[New12] demonstrated the power of optimized search and evaluation techniques, while more recent breakthroughs such as DeepMind’s AlphaZero[SHS⁺18] have shown the potential of self-play and deep learning in achieving a never-dreamed-of superhuman performance. Classical chess engines rely on complex and fine-tuned hand-crafted evaluation functions and high-speed search algorithms, whereas modern solutions try to replace as many human-crafted components with machine learning alternatives.

I have been fascinated by chess since my grandfather first taught me the basics. I have never been a strong enough player, so I wanted to combine my chess interest with my computer science knowledge that I have accumulated during my three years at university, as the vast domain of chess programming encompasses multiple domains in programming, such as object-oriented programming, databases, machine learning, data structures, low-level optimizations, and front-end development.

1.2 Problem Statement and Objectives

The core challenge of this thesis is to demonstrate that a single developer, who only has basic chess knowledge, can build a fully functional chess engine from scratch. The engine should be able to play at a decent level while not relying on some highly complex hand-tuned evaluation routines. It will depend solely on a learned evaluation model.

Key objectives

- Leverage established engine optimization techniques: Implement popular engine enhancement techniques such as magic bitboards, caching and custom object representation to create an engine that has a high legal moves generation speed.
- Leverage established tree search optimization techniques: Implement popular search enhancement techniques such as iterative deepening, alpha-beta pruning and move ordering, to increase the depth of the search.
- Machine-learned evaluation: Develop a model that evaluates a board position based on the current position of all the pieces, eliminating handcrafted functions.

This approach aims to blend the best, well-established engine crafting techniques, while also trying to leave the most complex part to the computer to figure out.

1.3 Research questions

To guide our investigation, we pose the following research questions, each of which will be addressed quantitatively via benchmarks and qualitative analysis:

1. What is the effect of each engine optimization technique on the end performance?
2. How is every search optimization effecting the game tree?
3. Is a machine-learned evaluator trained to predict board evaluations solely on already evaluated boards enough to create a formidable opponent?
4. What are the trade-offs between different model complexity and inference cost during search, and how do these trade-offs effect the engine performance?

1.4 Scope and Limitations

Scope This thesis focuses on building a minimal yet competitive chess engine: it implements FEN parsing, move generation, optimized move search, position evaluation, and a lightweight front-end that allows the user to play against the engine. The neural evaluator trained on a limited PGN dataset and using only FEN-derived input replaces the highly optimized and complex evaluation functions.

In the beginning, the model training and training data processing were done on a single machine, but as the tasks became more complex and more resource-intensive, the training was moved to the cloud, on the "Google Colab" platform. While it was possible to use the cloud to speed up the training, the use of the trained model could only be done on a local machine.

The speed and correctness of the engine's move generation are assessed via test suites and will be explained in more detail later. The quality of the moves and evaluation is analyzed by comparing the board evaluations of the model to a well-established engine named "Stockfish", on whose predictions the training was also based.

The engine will not use endgame tablebases or opening books, as I deemed it to take away from the model's ability to learn about every stage of the game.

1.5 Contributions

- **Standalone engine implementation:** A complete engine built from scratch, featuring FEN parsing, move generation (validated via perft), an optimized alpha-beta search loop and a minimal user interface.
- **Systematic optimization study:** An analysis of classical engine and search optimizations such as move ordering, transposition tables, magic bitboards and custom data structures, and their affect on search depth and nodes generated and checked per second.
- **Neural network evaluation model:** Design and training of a machine learning evaluator that consumes only FEN-derived input and predicts an board position evaluation.
- **Seamless hybrid integration:** A seamless use of the model inside the tree search loop, demonstrating the good cooperation between classic and modern chess programming concepts.

Declaration of Generative AI and AI-Assisted Technologies in the Writing Process
During the preparation of this thesis, the following tools were used: Overleaf's

built-in auto-correct and formulation helper TeXGPT, as well as OpenAI's ChatGPT, were used to assist in drafting, phrasing, and formatting. All output generated by these tools was carefully reviewed, edited, and approved by the author, who takes full responsibility for the accuracy, integrity, and final content of this work.

Chapter 2

Background and Literature

2.1 Chess as a Computational Problem

2.1.1 Rules and Notation

Board, Pieces and Objective Chess is a two-player board game played on an 8×8 grid of alternating light and dark squares [dEF21]. Each player, one using dark pieces and one using light pieces, starts with the same set of pieces: one king, one queen, two rooks, two bishops, two knights, and eight pawns.

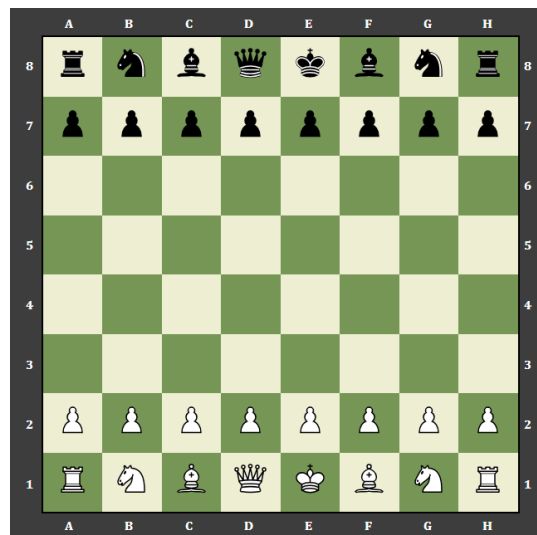


Figure 2.1: Chessboard coordinate system (files a–h, ranks 1–8).

The objective of the game is to place the opponent's king under checkmate, which means placing it under an inescapable threat of capture. The game can also end in a draw in multiple ways:

- A player remains without legal moves, but its king is not under attack, this is called stalemate.

- The same moves have been repeated for three or five times, this is called threefold or fivefold repetition.
- If no piece has been captured or no pawn was moved in the last fifty or seventy-five moves.
- If checkmate is impossible, for example if only the kings remain on the board.
- The players can also agree on a draw to end a game.

Piece Movements Each piece has its well-defined move set, which includes the way it moves and captures other pieces. Some pieces also have a special, unique behavior.

- **Pawn:** It can only advance one square forward and captures one square diagonally forward. The only exception is its first move, when it can move two squares forward. When a pawn reaches the opponents back rank (8th for white, 1st for black), it is immediately replaced, promoted, to any other piece (queen, rook, bishop or knight). Another special move is the *En Passant*, it is a capture move done by a pawn against an opponent pawn. It is available directly after an opposing pawn moves two squares. The friendly pawn can capture it "as if" the opposing pawn only moved one square.
- **Knight:** Moves in an *L* shape, two squares in one direction and then one perpendicular, and is the only piece that can jump over pieces, friendly or opposing, that come in its way. Captures on its landing square.
- **Bishop:** Slides any number of squares diagonally, capturing the first opposing piece it encounters.
- **Rook:** Slides any number of squares along ranks or files (rows or columns), capturing the first opposing piece it encounters. The rook has a special move named *Castling*, it is executed together with the king and can only be done if its the first move that they both execute and the king is not under attack during the move. It is separated in two subcategories: *Kingside* and *Queenside*. During this move, the king moves two squares towards the rook and the rook jumps to the other side of the king.
- **Queen:** Combines the move-sets of the rook and bishop, sliding along ranks, files or diagonals, while capturing the first opposing piece it encounters.
- **King:** Moves one square in any direction, capturing in the same manner. It can never move onto a square that is under attack by an enemy piece, i.e.

move into check. During a *check*, the player is forced to escape it by moving the king to a safe square or by stopping the attack (by capturing the attacker, or by moving a piece in the way of the attack).

Notation There are multiple ways of representing the game of chess. The ones that we used were:

- **Algebraic notation:** It is used to represent moves in a short string. Each piece has a corresponding letter (*K* for king, *Q* for queen, *N* for knight, *B* for bishop, *R* for rook, *P* for pawn), each rank has a corresponding number (1 to 8) and each file has a corresponding letter (*a* to *h*). To represent a move you take the moving piece and the destination square (for example *Be5*: bishop moves to *e5*). If the move is a capture, an *x* is placed between the piece and square, for checks *+* is added at the end, for mate *#* and castles are represented by *O-O* and *O-O-O*, king- and queenside respectively.
- **Forsyth–Edwards Notation (FEN):** Is used to represent an entire board. Each piece has an associated letter, just like in the *Algebraic notation*, but they are uppercase if they are a white piece and lowercase if they are a black piece. It also contains the current move-maker, the castle rights for both sides, en passant pawn position, half-move clock and Full-move number (the last two are not important for this project). An example of a FEN string would look like this:

`rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1`

(The 8 each represent eight empty tiles and the / divide the ranks)

- **Portable Game Notation (PGN):** This is used to store games. It contains each move, some information about when the game took place and some information about the players. The details are not so relevant for this project.

2.1.2 Computational Complexity

One of the main challenges is the huge number of possible games and positions. Chess is *EXPTIME*-complete[FL81], which means that it takes time exponential in relation to the board size and move limit, to determine the true outcome of every move. Claude Shannon famously estimated that the lower bound of the *game-tree complexity* of chess is about 10^{120} , based on an average of 35 move possibilities after each move. Here *game-tree complexity* refers to all possible move sequences. He also estimated that there are 10^{47} possible board configurations.

Measure	Estimate	Source
Game-tree complexity	10^{120}	Shannon (1950)[Sha50]
State-space complexity	10^{47}	Shannon (1950)[Sha50]
EXPTIME-completeness proof	-	Fraenkel & Lichtenstein (1981)[FL81]

Table 2.1: Key complexity estimates for chess.

This huge number of possibilities leads to multiple innovative techniques that seek to reduce the size of the search tree. Alpha-Beta pruning is one of the best-known techniques; in its best case, it can reduce the branching factor from b to almost $\sqrt{b}$, cutting the search complexity from $O(b^d)$ to about $O(b^{d/2})$ [KM75]. Transposition tables[BU02] cache previously encountered positions to skip identical sub-trees and avoid redundant exploration.

2.1.3 Performance Metrics

To evaluate the correctness and the strength of the engine, several metrics were used during the development of the chess engine. All of these combined will give a full picture of the correctness and overall playing ability.

- **Perf (Performance Test):** Counts the number of leaf nodes generated at a given search depth. It is used to verify the correctness of the move generation, that means check if no illegal moves are allowed or legal moves are skipped.
- **Nodes per second (NPS):** Measures the speed of the engines move generation. To measure this we take the results of the perft and divide it by the time it took to execute the test.
- **Search Depth:** Records the maximum plies reached during search within a specified time limit. It is used to see how far into the future the engine managed to look.
- **Centipawn Loss (CPL):** Represents the average difference between the two sides. A positive value means that the white side has the advantage and a negative value means that the black side has the advantage.
- **ELO Proxy:** Estimates the strength of a player. A beginner has around 600 ELO, an experienced player has about 1400, a good player is about 2000 and a grandmaster is about 2500.

Metric	Definition / Formula
Perft	# leaf nodes at depth d (correctness)
NPS	$\frac{perft(d)}{time(d)}$ (speed)
Search depth	Maximum plies reached in time budget
CPL (centipawn loss)	Avg. evaluation error vs. ground truth (in hundredths of a pawn)
ELO proxy	Estimated from win/draw/loss statistics

Table 2.2: Summary of performance and strength metrics.

2.2 Classical Search Algorithms

2.2.1 Minimax Algorithm

The **Minimax algorithm**[Bre96] is the fundamental decision-making algorithm that is used to "solve" games that can be represented as a tree. Some of these games are: *Checkers*, *Connect Four*, *Shogi* and *Chess*. The main attraction of this algorithm is its simplicity, but this simplicity is also its biggest downside. For games like *Chess* where the game-tree is extremely big, this algorithm requires too much time and too many resources to be viable.

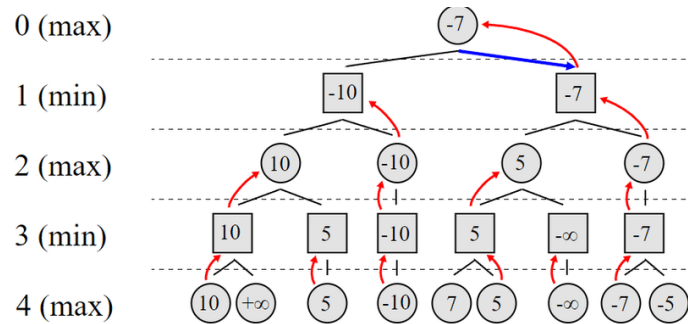


Figure 2.2: Example game-tree illustrating a minimax search[Men22].

The algorithm is a recursive search algorithm rooted in game theory, designed to simulate every possible move sequence and to find the one that leads to the best outcome for the current player, while assuming that both players act optimally. It evaluates the entire game tree to a predefined depth, applying an evaluation function at each node, which means it has to evaluate a tree of size $O(b^d)$, where b is the branching factor and d is the search depth. After scoring the nodes, it chooses the highest value when it would be the turn of the current player (the engine in our case) and the lowest value when it would be the turn of the opponent (the human in our case).

Algorithm 1 Minimax Algorithm - a baisc implementation

```

function minimax(node, depth, maximizingPlayer)
  if depth = 0 or node is terminal then
    return evaluation(node)
  end if
  if maximizingPlayer then
    maxEval  $\leftarrow -\infty$ 
    for child in node.children do
      eval  $\leftarrow$  minimax(child, depth - 1, false)
      maxEval  $\leftarrow$  max(maxEval, eval)
    end for
    return maxEval
  else
    minEval  $\leftarrow +\infty$ 
    for child in node.children do
      eval  $\leftarrow$  minimax(child, depth - 1, true)
      minEval  $\leftarrow$  min(minEval, eval)
    end for
    return minEval
  end if
    
```

2.2.2 Alpha-Beta Pruning

The **Alpha-Beta pruning** algorithm[KM75] significantly enhances the efficiency of *Minimax* by eliminating (pruning) branches of the tree that are clearly not going to affect the final decision, reducing the size of the game tree while keeping decision optimality. This behavior is achieved by having two parameters, *alpha* α and *beta* β . At a maximizing node, we iterate through the possible moves by calling the search with updated parameters. If the child's value is $\geq \beta$, we prune the remaining siblings (this is called β -cutoff). At minimizing nodes we do the exact opposite; if a child's value is $\leq \alpha$, we perform an α -cutoff.

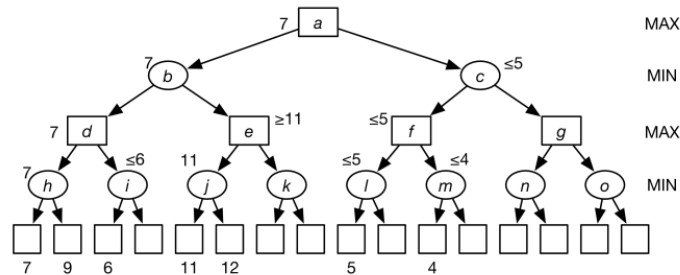


Figure 2.3: Example game-tree illustrating an Alpha-Beta search[PM10].

Knuth and Moore showed that in the best case, with perfect move ordering to maximize the cutoffs, we end up with a branching factor of $\sqrt{b}$, reducing the complexity from $O(b^d)$ to $O(b^{d/2})$. On average, alpha-beta achieves a branching factor

of approximately $O(b^{3d/4})$ [KM75].

Algorithm 2 Alpha-Beta Pruning Algorithm - basic implementation

```

1: function AlphaBeta(node, depth,  $\alpha$ ,  $\beta$ , maximizingPlayer)
2: if  $depth = 0 \vee$  node is terminal then
3:   return evaluation(node)
4: end if
5: if maximizingPlayer then
6:    $value \leftarrow -\infty$ 
7:   for child in node.children do
8:      $value \leftarrow \max(value, \text{AlphaBeta}(child, depth - 1, \alpha, \beta, \text{false}))$ 
9:      $\alpha \leftarrow \max(\alpha, value)$ 
10:    if  $\alpha \geq \beta$  then
11:      break  $\triangleright \beta$ -cutoff
12:    end if
13:  end for
14:  return  $value$ 
15: else
16:    $value \leftarrow +\infty$ 
17:   for child in node.children do
18:      $value \leftarrow \min(value, \text{AlphaBeta}(child, depth - 1, \alpha, \beta, \text{true}))$ 
19:      $\beta \leftarrow \min(\beta, value)$ 
20:    if  $\beta \leq \alpha$  then
21:      break  $\triangleright \alpha$ -cutoff
22:    end if
23:  end for
24:  return  $value$ 
25: end if

```

2.2.3 Search Enhancements

To be able to handle the exponential growth of the game tree and boost search efficiency, modern engines layer several well-studied enhancements on top of basic alpha-beta. In this section, we review four cornerstone techniques: move ordering heuristics, Zobrist hashing with transposition tables, and iterative deepening.

Move Ordering Alpha-beta pruning’s power depends critically on examining the most promising moves first. Good move ordering maximizes early cutoffs, reducing the effective branching factor. Common heuristics include:

- *Principal Variation (PV) Move*: Try last iteration’s best move first.
- *Capture Ordering*: Score captures by “most valuable victim - least valuable attacker: (MVV-LVA).

- *Killer Moves*: Remember moves that caused beta-cutoffs at the same depth in other branches and try them early.
- *History Heuristic*: Rank non-capture moves by how often they have caused cutoffs in the past.

By combining these, engines achieve move-ordering quality that drives alpha-beta toward its best-case $O(b^{d/2})$ complexity.

Zobrist Hashing and Transposition Tables Many positions arise via different move sequences (transpositions). Zobrist hashing[Zob90] assigns each piece-on-square combination a random 64-bit key; by XORing these, the engine obtains a nearly unique key for every position. A fixed-size transposition table uses these keys to store previously searched positions along with their best evaluation, best move, node type (exact, lower bound, upper bound), and search depth. On encountering a hashed position:

- If the stored depth is at least as great as the current search depth and the node type is *Exact*, the engine can directly reuse the stored score, avoiding any subtree search.
- If the node type is *Lower bound* and the stored score exceeds or equals the current β , it yields a cutoff without further search.
- If the node type is *Upper bound* and the stored score is less than or equal to the current α , it similarly produces a cutoff.
- Otherwise, the entry is considered insufficient for the current window and is updated after the search completes.

This mechanism not only prunes redundant work but also provides high-quality move-ordering information across the tree, using previously explored lines to accelerate deeper searches.

Iterative Deepening Rather than jumping directly to a target depth, the engine performs a series of depth-limited searches at increasing depths (e.g. 1, 2, 3, ..., d). Iterative deepening[Kor85] provides natural time management - after each iteration, the engine can decide whether to continue or return the best move found so far. Without the use of *Transposition Tables*, this technique could not be achieved, as it would require a lot of recalculations of already evaluated positions.

2.2.4 Board Representations

Chess engines must choose an internal data structure to represent the board state and piece locations. Early engines often used a simple *mailbox* or *array* structure: a one-dimensional array of 64 entries, each holding a code for the occupying piece or empty. This approach is easy to implement and debug but can suffer from poor cache performance and requires bounds checks on every move-generation offset. An alternative is the *piece-list* representation, where each side maintains a list of its active pieces with their square indices. It reduces scanning of empty squares but complicates attack detection, since one must iterate over every piece and then compute its moves.

The most common modern format is the *BitBoard*: a 64-bit integer in which each bit corresponds to one square on the 8×8 board. One or more BitBoards encode occupancy for each piece type and/or side. BitBoards enable extremely fast operations: attacks, occupancy tests, and sliding-piece ray attacks, via a few bitwise operations. A pretty common variant is *magic* BitBoards (for constant-time sliding attacks). Although BitBoards require careful masking to avoid off-board moves, they generally deliver the highest throughput for move generation and position evaluation.

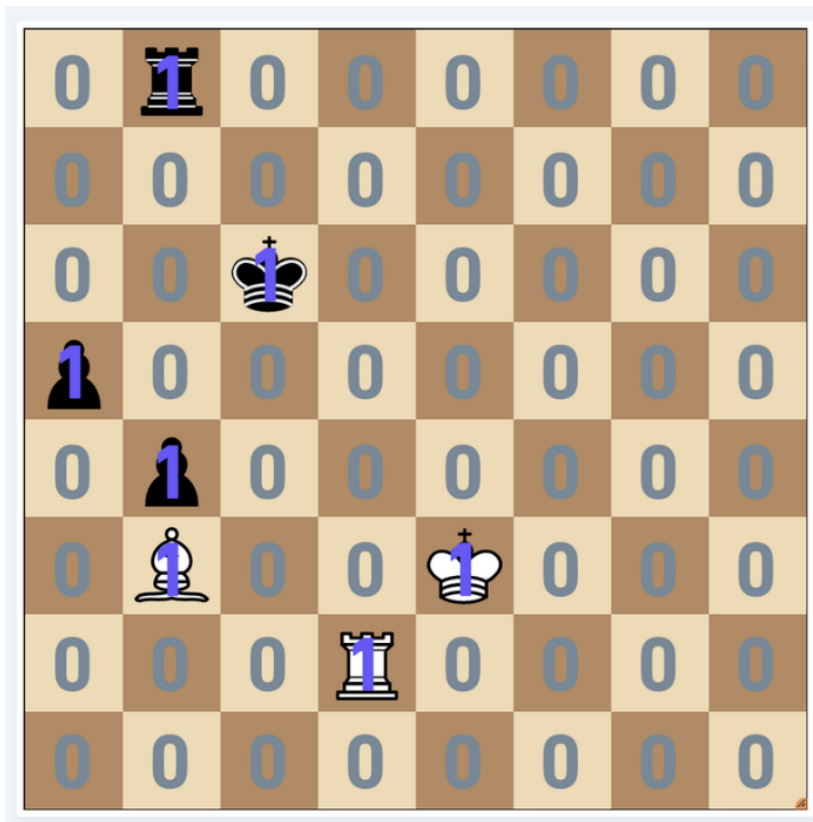


Figure 2.4: Position with pieces overlaid with a bitboard that indicates the presence of pieces[lik]

Rep.	Memory	Move Gen Speed	Cache-friendliness
Mailbox	Low	Low	Poor
Piece-list	Medium	Medium	Fair
BitBoard	Low	Very High	Excellent

Table 2.3: Trade-offs between board representations.

2.2.5 Move-Generation Techniques

Move generation usually proceeds in two stages. First, each piece type produces all *pseudo-legal* moves based solely on its movement rules and board occupancy. Pseudo-legal moves are those that follow the piece’s allowed patterns (for example, a bishop’s diagonal steps or a knight’s L-shaped jumps) and avoid landing on squares occupied by the same side, but they do not yet check whether the king would be left in or moved into check. Second, the engine filters out any of these pseudo-legal moves that would expose the king to attack or otherwise violate special constraints (pins, castling rights), yielding the final set of fully legal moves.

For non-sliding pieces (knight, king, and pawn captures), move generation is often implemented with precomputed *offset tables*: fixed arrays of relative square indices. The engine applies each offset to the piece’s current square, checks for board boundaries, and then classifies the destination as a normal move or capture/attack based on occupancy BitBoards[AVAB⁺07].

Sliding pieces (bishop, rook, queen) require scanning along each ray until blocked. A straightforward loop-based approach steps one square at a time, stopping at the first occupied square. More advanced engines use *Magic BitBoards*: by masking the occupancy along each ray and multiplying by a precomputed “magic” constant, one obtains an index into a table of attack BitBoards in constant time, eliminating loops.

Approach	Complexity	Ops per move	Cache-friendliness
Offset tables\$ knight/king/pawn)	$O(1)$	$\approx 10\text{--}20$	Excellent
Loop-scan sliding\$ rook/bishop/queen)	$O(d)$ per ray	$\approx 50\text{--}100$	Fair
Magic BitBoards	$O(1)$	$\approx 5\text{--}10$	Good

Table 2.4: Comparison of move-generation techniques.

Finally, special moves such as castling, en passant, and pawn promotions are handled with additional rules: castling legality is checked via attack BitBoards and castling-rights flags; en passant captures use a simple flag for the square eligible for such a move; and promotions expand the pawn’s capture and advance moves into

multiple promotion-move entries (a new move for each possible promotion type). By combining these techniques, a modern engine can generate and validate all legal moves in just a few dozen instructions per move, achieving node-generation rates in the millions per second.

Magic BitBoards Magic BitBoards[Kan07] leverage precomputed tables and special “magic” constants to generate sliding-piece attacks in constant time. For each square and sliding direction, a 64-bit mask isolates the relevant blockers (enemy/friendly pieces) on the ray; multiplying this masked occupancy by a carefully selected magic number and then shifting the result yields a unique index into a table of attack BitBoards. Each table entry encodes all legal destinations for that blocker configuration. The magic numbers are chosen so that every possible masked occupancy produces a distinct index, eliminating collisions. By replacing loop-based ray scans with a few bit-wise operations and a single array lookup, magic BitBoards deliver dramatically faster and more predictable sliding move generation.

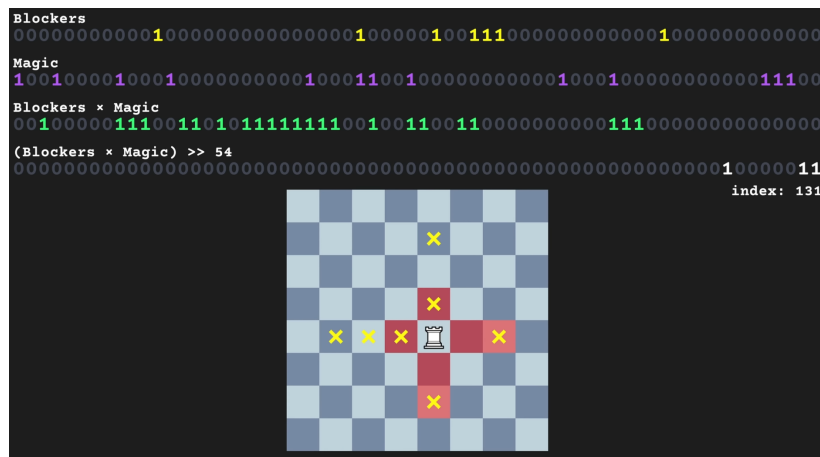


Figure 2.5: Magic BitBoard workflow: blocker mask $\rightarrow$ magic multiplication $\rightarrow$ index.

Magic Number Generation Finding a suitable magic constant for each square and sliding direction is an offline, brute-force process. For a given ray (e.g. rook on the a-file), we first enumerate all possible blocker configurations (bit-masks indicating which squares along that ray are occupied). Each configuration has a corresponding "attack set", the set of legal destination squares. We then repeatedly generate random 64-bit candidates with a bias toward sparse bit patterns (as many "0" bits as possible, to mix as few bits as possible when multiplying masks), and for each candidate, we test whether the following formula maps every distinct blocker mask to a unique index into an attack-table of size 2^{bits} .

$$index = blockers \times magic \gg (64 - bits) \quad (2.1)$$

A magic constant is accepted only when no two different blocker masks collide under this hashing. Although finding a collision-free magic can require millions of trials, doing so once per square produces compact, perfectly indexed lookup tables that enable constant-time sliding-piece move generation.

Algorithm 3 Generation of magic numbers

```

1: generate all blockerMasks for this square
2: repeat
3:   magicCandidate  $\leftarrow$  randomSparse64()
4:   clear collisionMap
5:   for mask in blockerMasks do
6:     idx  $\leftarrow$  (mask  $\times$  magicCandidate)  $\gg$  (64 - bits)
7:     if collisionMap[idx] exists then
8:       break
9:     end if
10:    collisionMap[idx] = mask
11:  end for
12: until no collisions
13: return magicCandidate

```

2.2.6 Material and Positional Factors

In classical chess engines, the evaluation function is a weighted sum of material and positional factors. Material balance assigns fixed values to each piece type (e.g. pawn = 1, knight/bishop = 3, rook = 5, queen = 9), while positional factors draw on heuristics such as piece-square tables, pawn structure (isolated, doubled, passed pawns), king safety (pawn cover, enemy attacker proximity), mobility (number of legal moves), and control of key squares or files. Although these hand-tuned evaluation terms can yield strong play, this thesis does not implement a handcrafted evaluator; it is discussed here only to illustrate the conventional approach before introducing our machine-learned alternative.

2.3 Machine-Learning Foundations

2.3.1 Machine-Learning Overview

Machine learning (ML) is a branch of artificial intelligence (AI)[Alp21] that enables computers to learn patterns and make predictions from data without being explicitly programmed for each task. At its core, ML seeks to approximate an unknown function

f by fitting a parameterized model $\hat{f}(x; \theta)$ to a collection of examples $\{(x_i, y_i)\}_{i=1}^N$, where x_i is an input signal and y_i the corresponding output.

Learning Paradigms ML methods are commonly categorized by the type of feedback available during training:

- *Supervised learning*: uses labeled data (x, y) to learn mappings from inputs to outputs. Regression predicts continuous values (e.g. evaluation scores), while classification predicts discrete labels (e.g. game outcomes).
- *Unsupervised learning*: discovers structure in unlabeled data, via clustering or dimensionality reduction, without explicit targets.
- *Reinforcement learning*: trains agents to make decisions by maximizing rewards gained through trial and error, as exemplified by self-play in chess engines[ZY20].

Model Training Training a model involves three key components:

1. **Model architecture**: the choice of function class (e.g. neural networks, decision trees, support vector machines) and its parametrization θ .
2. **Loss function**: a measure $L(y, \hat{y})$ of the discrepancy between model predictions $\hat{y} = f(x; \theta)$ and true targets y . Common choices include `mean_squared_error` for regression and `cross-entropy` for classification.
3. **Optimization algorithm**: an iterative procedure (e.g. gradient descent, Adam) that adjusts θ to minimize the expected loss on the training data, often with regularization terms to prevent overfitting.

Generalization, Evaluation and Overfitting A main goal is to achieve good performance on unseen data as well. To do that, the model needs to avoid overfitting. Overfitting means that a model learned the training data too well and it fails to capture an underlying pattern. This can happen by having a very complex model and too little training data. In order to have a complex model while not having to collect too much data, techniques such as `early stopping`, where the training stops automatically when an overfitting threshold is reached, `dropout`, which disables a random part of the neural network during training, and `L2 weight decay` which adds $\lambda \|\theta\|^2$ to the loss, penalizing large weights, were created.

2.3.2 Neural Network Foundations for Chess Evaluation

Supervised Regression for Position Evaluation Chess-position evaluation can be cast as a supervised learning task: given a representation x of a board state and a ground-truth score y (a centipawn advantage provided by a strong engine, in our case *Stockfish*), learn a function $f(x; \theta)$ parameterized by θ that approximates $y \approx f(x; \theta)$. Because evaluation values are continuous, this is a regression problem [GG21]. A common choice of training objective is the *mean squared error* (MSE), which penalizes large deviations quadratically.

$$\text{MSE}(\theta) = \frac{1}{N} \sum_{i=1}^N (y_i - f(x_i; \theta))^2, \quad (2.2)$$

2.3.3 Feature Representation

A very important part of every model is the format of the encoding of the input data. For chess, there are multiple ways of doing this, but the two main ones are `BitBoards` and `One-Hot Encoding`. They both are based on bit representations. The `BitBoards` representation is the same as the one in the engine, and the `One-Hot Encoding` representation involves using a binary vector where only one bit is ‘hot’ (set to 1) to represent the presence of a piece in a specific position, ensuring a unique and unambiguous representation. Along with the piece representation, a small vector with flags representing side-to-move, castling availability, and en passant targets is also used. These channels form a tensor $x_{\text{board}} \in \mathbb{R}^{8 \times 8 \times 12}$ and an auxiliary vector $x_{\text{extra}} \in \mathbb{R}^d$. Such structured inputs allow convolutional layers to extract spatial patterns of piece interactions while integrating global game-state flags.

2.3.4 Neural Network Architectures

Two main types of neural networks are used in the chess programming field for position evaluation:

Convolutional Neural Networks (CNNs) Convolutional layers apply small, learnable filters (commonly 3×3) across the input to detect patterns in local sections of the board. Because the same filters are reused at every location, the number of parameters stays manageable. By stacking several convolutional layers, the network’s *receptive field* grows, allowing it to combine simple local features into more complex, larger-scale positional concepts [ON15].

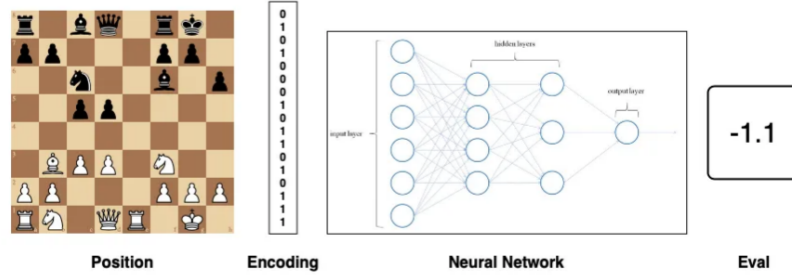


Figure 2.6: A mock diagram of a Convolutional Neural Network architecture.[Spe21]

Residual Networks with Attention Residual networks (ResNets) are able to capture long-range dependencies, such as threats or coordination across opposite sides of the board. While they can produce more accurate evaluations, they also require a lot more resources for development, training, and inferences. Because of the hardware limitations and the need to run thousands of inferences per second, in order to be able to look multiple moves in the future, this type of architecture will not be used.

2.3.5 Huber Loss Function and Sample Weighting

The Huber [GG21] loss function is designed to be less sensitive to outliers in the data than the standard mean squared error. A combination of Huber loss with sample weighting ensures that the model pays extra attention to the parts of the data where predictions are close to the true values (small errors), while also being less influenced by extremely incorrect predictions (outliers). The sample weighting adds an additional layer of focus, particularly emphasizing certain conditions such as balanced or drawn chess positions.

$$L_{\delta}(e) = \begin{cases} \frac{1}{2}e^2 & |e| \leq \delta, \\ \delta(|e| - \frac{1}{2}\delta) & \text{otherwise,} \end{cases} \quad e = y - \hat{y}, \quad (2.3)$$

2.3.6 Optimization Algorithms

To train the network, an adaptive optimizer called *Adam* is used. *Adam* keeps two running averages for each parameter:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla_{\theta} L, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla_{\theta} L)^2, \quad (2.4)$$

where m_t tracks the mean of recent gradients and v_t tracks their uncentered variance. Each parameter update then takes the form

$$\theta_t = \theta_{t-1} - \eta \frac{m_t}{\sqrt{v_t} + \epsilon}, \quad (2.5)$$

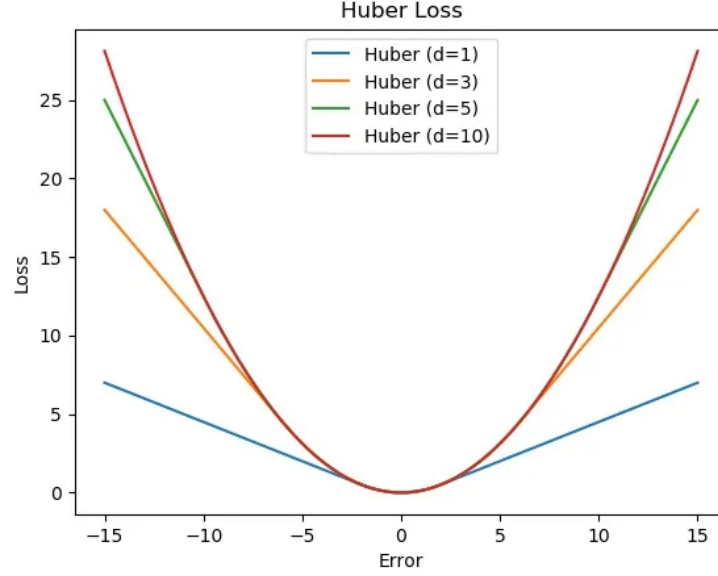


Figure 2.7: Huber Loss Function. ($d = \delta$)[Vid20]

so that learning rates are effectively scaled per parameter based on gradient history.

To keep training stable, gradients are clipped to a maximum norm, preventing any single update from becoming too large. A learning-rate schedule reduces η when the validation loss plateaus, ensuring gradual fine-tuning.

2.3.7 Evaluation Metrics

Model performance is measured by:

- *Root-mean-square error (RMSE)* and *mean absolute error (MAE)* on a held-out validation set.
- *Correlation* with engine evaluations, indicating how closely the model tracks its teacher.

These metrics quantify both raw prediction accuracy and alignment with established evaluation standards.

2.4 Benchmarking

2.4.1 Perf Testing

Perft, short for "performance test" or "perftest", is a standard method for both validating move generation correctness and measuring raw search speed. A perft call for a depth d explores every possible move sequence for length d from a given position and returns the total number of leaf nodes reached.

History and Evolution Perft was first implemented in the COBOL program RSCE-1 by R.C.Smith. It was submitted to the USCF for evaluation, and it was covered in a 1978 Computerworld article[Com78, Smi78]. RSCE-1 was designed to perform position analysis and forced-mate searches up to three plies; its perft(3) from the initial position was reported as 8,902. In 1995, Steven Edwards proposed a move-path enumeration method in the Spector engine and has since maintained Perft benchmark suites. The term “perft” itself appears to have been popularized by a Crafty command, even though its early behavior differed slightly[Bri00]. In August 2017, Ankan Banerjee used a hybrid CPU/GPU approach on an NVIDIA DGX-1 cluster to compute $\text{perft}(15) = 2.015.099.950.053.364.471.960$ [Ban17].

Algorithm 4 Recursive Perft Implementation

```

1: function PERFT(node, depth)
2:   if depth = 0 then
3:     return 1
4:   end if
5:    $total \leftarrow 0$ 
6:   for all move in GENERATEMOVES(node) do
7:     MAKEMOVE(node, move)
8:      $total \leftarrow total + \text{PERFT}(node, depth - 1)$ 
9:     UNMAKEMOVE(node, move)
10:  end for
11:  return  $total$ 
12: end function

```

Correctness Validation There are a lot of benchmark positions with well-known perft values, so the best way to analyze the correctness of an engine is to compare its values to the already defined ones. Typical tests range from depths 1 to 6, as anything deeper becomes too time-expensive.

Performance Validation The cleanest and easiest way to measure the performance of an engine is to time the execution of perft. After timing the execution to a predetermined depth d , we can use the following formula to determine the speed of the engine’s move generation, measured in *nodes per second* (NPS).

$$NPS = \frac{\text{perft}(d)}{\text{time}(d)} \quad (2.6)$$

Chapter 3

Engine design

3.1 Overview and Goals

Chess programming is a very vast and complex domain. The development of an entire engine from scratch can be a very intimidating challenge. There are a lot of decisions to be made when starting, and numerous ways to reach the same goal. The main goal of the developmental phase was to create a decently performing engine, that means reaching a speed of about *1.000.000 NPS* while executing the *Perft* tests, that uses multiple optimization techniques. These would be added one by one to shed light on their benefits and developmental challenges.

This chapter will focus on the evolution, core ideas, and techniques of the chess engine. It will start with the original, unoptimized engine and gradually better design ideas will be implemented, while analyzing the performance benefits and developmental challenges that come with them.

3.2 Project Structure and Tools

The project features a frontend and backend that communicate via HTTP services.

The backend engine is implemented in Java using Spring Boot and is structured following the classic *Model-Service-Controller* pattern. Maven was used for the management of dependencies and the build lifecycle, while Lombok reduces boilerplate code by generating getters, setters, and constructors. Java is not the fastest platform when compared with C and C++ for raw performance, but the small performance penalty is outweighed by the faster development cycle and my extensive familiarity with the language.

The frontend is built with Angular and styled using PrimeNG components. It is not something this thesis is focusing on, so the only objective regarding the frontend was to create a simple board, that allowed a user to move pieces and after that

displayed the moves the model was making in response.

Backend Package Structure

- **controllers:** contains the REST controllers used to connect the backend to the frontend.
- **exceptions:** contains the custom exception class and codes, used to create a more detailed explanation of things that could go wrong during runtime.
- **models:**
 - **bitboards:** contains everything that is relevant to bitboards.
 - **board:** contains the core of the engine, the game board.
 - **dtos:** contains the data transfer objects.
 - **moves:** contains everything that is relevant to the custom representation of a move.
 - **pieces:** contains everything that is relevant to pieces, for example the generation of moves.
- **services:** contains the core engine logic, including:
 - **ChessValidator:** contains some basic validation methods.
 - **FenService:** contains the entire logic that handles FEN strings.
 - **GameService:** contains the logic that connects the controller to the actual engine.
 - **ModelService:** contains the logic that handles model predictions.
 - **MoveFinder:** contains the search and decision making algorithm.
- **utils:** contains utility classes used in the entire engine, making development easier and the code cleaner.
- **test:** the module that contains all *Perft* tests.

3.3 Performance and Correctness Tests via Perft

To provide a consistent baseline for both correctness and performance measurements throughout this chapter, the engine is evaluated using a suite of perft tests at each design phase. Perft (performance test) counts leaf nodes from a given position to a specified depth, making it an ideal tool for spotting move-generation bugs and

timing move-generation throughput under identical conditions. The same set of six reference positions, ran to depths of 4 through 6 (depending on the size of the tree),[Che25] were used repeatedly—first to validate correctness and then to quantify speed improvements as optimizations are introduced:

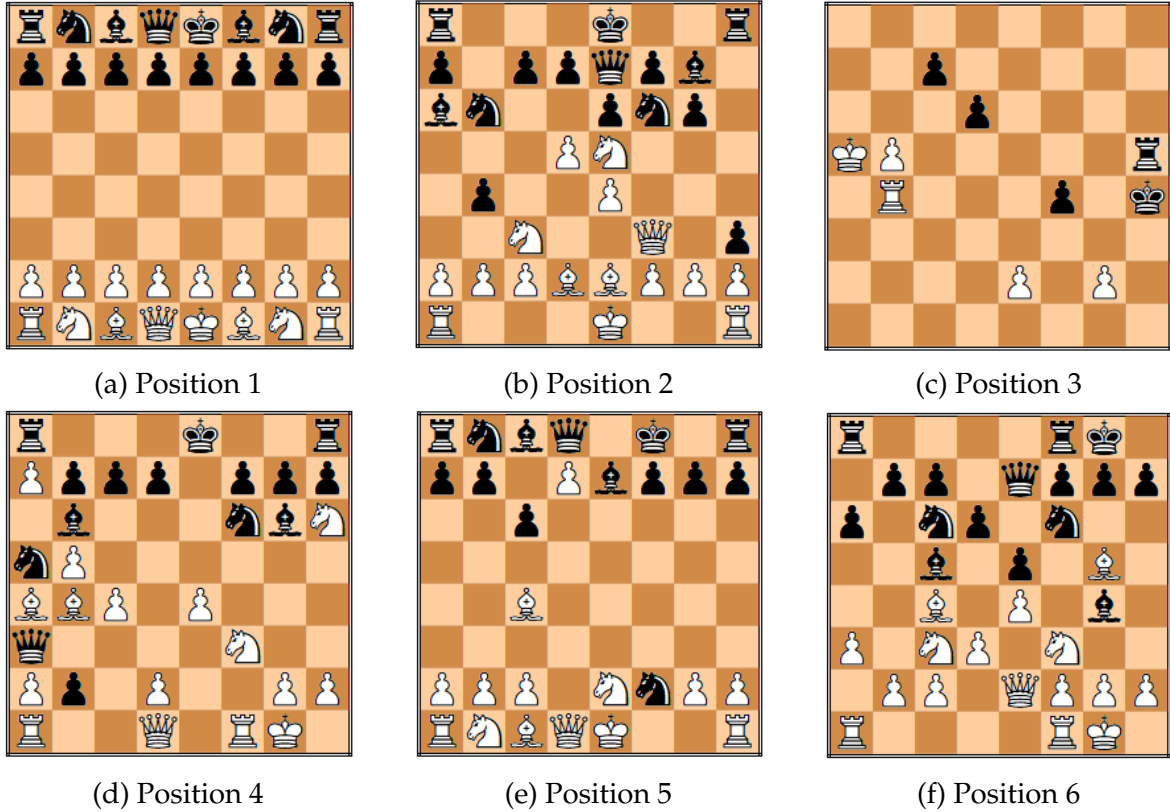


Figure 3.1: The six perft test positions that will be used.

These positions helped find all possible errors within move generation, especially in rare situations that would be too hard to manually test, for example, exposed checks, promotions, promotions following an attack, en passant, valid and invalid castling positions. Whenever my engine’s count did not match Stockfish’s reference value, a divide-and-conquer approach was used. The main tree was split into subtrees, each starting from one of the root’s children. This process was repeated until the missing or extra move was found.

3.4 Design Evolution

3.4.1 Phase I: OOP and Immutable State

Board and Piece Representation In the initial prototype, a *mailbox* representation was used. A `Board` object contained an array of 64 `Tile` instances, each identified by an index and each holding a reference to a `Piece` or `null` if it was empty. Beyond

the board configuration, the `Board` class also tracked all legal moves for each side, the position of the en passant pawn (if any), and castling rights. Every piece type had its own class and inherited the same `Piece` superclass that encapsulated properties like alliance (white or black), current position, and an `isFirstMove` flag to help handle double pawn advances and castling moves.

Move Representation Every chess move was represented by a `Move` object that bundled the following properties: starting and destination position, the move type (normal, double-pawn-advance, attack, promotion, etc.) to help distinguish special cases, and a string (e.g. "Q", "R", "K", "B") that represented the type of promoted piece, if a promotion move was executed. By storing every relevant piece of information about a move in a single object, the engine can handle the move execution and information transfer between front- and back-end very clearly.

Move Generation For the move generation, each piece had a `moveGeneration()` method that created a list of all pseudo-legal moves for the selected piece. Internally, each piece relied on a set of predefined offsets that were applied to its current position.

- **Knight, King:** These pieces had the simplest and most straight forward move generation algorithm. The `King` used the offsets $\pm 1, \pm 8$ and the `Knight` used the offsets $\pm 6, \pm 10, \pm 15, \pm 17$. Their algorithm applied each offset and checked the candidate destination. If the destination was empty, it created a normal `Move` object, if it was occupied by an opponent, an `attack Move` object, and if its not a valid destination (for example its outside the board bounds), the offset was skipped.
- **Sliders (Queen, Rook, Bishop):** Their offsets represented direction vectors ($\pm 1, \pm 8$ for rooks, $\pm 7, \pm 9$ for bishops and all eight for queens). For each direction, the engine stepped repeatedly and added a move, until it a blocker was found or the board edges were found.
- **Pawn:** Pawn move generation was the most complex as there were multiple checks to be done. For its first move, a pawn can do a normal advance or a double advance can be done; it can perform an en passant attack right after the opponent executes a double pawn advance; and it can perform promotions and promotions after an attack. All these scenarios needed to be covered, making this method the most complex one.

After generating all pseudo-legal moves of all pieces on the board and putting them in a list, the items of the list were filtered. to remove the moves that resulted in a friendly check. This was done by executing every move and checking if the friendly

king was in check. After this, the castle moves were generated and added to the legal moves list.

Algorithm 5 Pseudo-legal Queen sliding moves with directional offsets & loop scan

```

1: function GENQUEENSLIDESLOOP(occupancy, ownPieces, queenSquares, direc-
   tions)
2:   moves  $\leftarrow \{\}$ 
3:   for all sq  $\in$  queenSquares do
4:     for all dir  $\in$  directions[Queen] do
5:       cur  $\leftarrow$  sq + dir
6:       while ONBOARD(cur) do
7:         if ownPieces  $\wedge$  (1 $\ll$ cur)  $\neq$  0 then
8:           break
9:         end if
10:        moves.APPEND(MOVE(sq, cur))
11:        if occupancy  $\wedge$  (1 $\ll$ cur)  $\neq$  0 then
12:          break ▷ capture stops ray
13:        end if
14:        cur  $\leftarrow$  cur + dir
15:      end while
16:    end for
17:  end for
18:  return moves
19: end function

```

Move Execution The object-immutability principle was followed during the phase. Instead of mutating the existing `Board` instance, a new one was created every time a new move was applied. This approach made every step of the process easy to implement, and undoing moves did not require complex additional logic since a board could simply be discarded, allowing the previous one to be used again.

Takeaways and Performance The development process of this approach was easy and straightforward, as it did not require any extremely complex logic. The code was clean and the execution process of every operation was easy to understand as everything was clearly organized in objects with clear properties and methods. Every time a move was executed or the state of some object needed to be changed, simple operations needed to be done. While everything went smoothly, the performance test showed just how inefficient this approach was:

Table 3.1: Perf Results for Phase I Engine

Position	Depth	Leaf Nodes	Execution Time (s)
Position 1	5	4,865,609	74
Position 2	4	4,085,603	77
Position 3	6	11,030,083	120
Position 4	5	15,833,292	319
Position 5	4	2,103,487	38
Position 6	4	3,894,594	67

After analyzing the main bottlenecks of the engine, it was clear that the immutability of every object and the lack of any caching were causing a huge performance problem. The engine only managed to reach a speed of about *63.000 NPS*, which was far from the original goal of *1.000.000 NPS*. The main problem was that every time a move was made and a new board was created, the legal moves for both sides needed to be recalculated from scratch. This meant that every tile needed to be checked again for an occupying piece and its moves needed to be calculated again. This was not a big issue for pieces like the king and knight, but for sliding pieces like the queen, this process caused a big problem. The only thing that helped the performance was the fact that undoing a move only required going back to an already existing object and discarding the current board instance. These results clearly indicated that the repeated full-search operations, the lack of caching, and the over-reliance on creating new objects were limiting the engine from being usable in any other context than local player versus player games. Nonetheless, it was a good starting point for the development process; it offered good insight into the overall architecture of the engine, and it clearly showed the main performance bottlenecks.

3.4.2 Phase II: Move Generation with BitBoards and Caching

In Phase II, the goal was to replace the move generation algorithms of every piece with algorithms based on BitBoards and precalculated move patterns. In order to skip the need for recalculations after every move, every possible move for every piece from every tile needed to be calculated and cached. Objects were too expensive to store in such large amounts, so BitBoards needed to be used to represent the pieces of the board.

The first step was to create a new `PiecesBitBoards` class and store it inside the `Board` class. It contained a `BitBoard` for every piece type and alliance. This replaced slow loops through the list of `Tile` objects with very fast bitwise operations. For example, to find the position of the black king, the corresponding `BitBoard` could be used, instead of looping over an entire list of objects.

The next step was replacing the move generation algorithm for every piece to use a new algorithm based on BitBoards and caching. To calculate and cache all moves, the same offset method was used for the *Pawn*, *King*, *Knight* because the number of possible moves from each tile was small enough that the look-up table would not require excessive memory. The sliding pieces (*Queen*, *Rook*, *Bishop*) represented a greater challenge since their legal moves depended on the pieces that occupied each direction. The number of possible occupations was far too great to generate classic attack meshes for every square; that is why *Magic BitBoards* were needed. This technique made it possible to have small look-up tables that contained all possible moves. The new algorithm took the current position of a piece, the precomputed moves for that square, and then used the `PiecesBitBoards` in order to decide which moves were attacks, which were blocked by friendly pieces, and which were legal.

Algorithm 6 Pseudo-legal Queen moves with Magic BitBoards

```

1: function GENQUEENMAGIC(occupancy, ownPieces, queenSquares, magicInfo,
   attackTable)
2:   moves  $\leftarrow \{\}$ 
3:   for all sq  $\in$  queenSquares do
4:     m  $\leftarrow$  magicInfo[sq].mask
5:     k  $\leftarrow$  magicInfo[sq].magicNumber
6:     b  $\leftarrow$  magicInfo[sq].relevantBits
7:     idx  $\leftarrow$  (((occupancy  $\wedge$  m)  $\times$  k)  $\gg$  (64 - b))
8:     bb  $\leftarrow$  attackTable[sq][idx]  $\wedge$   $\neg$ ownPieces
9:     for all dest  $\in$  Bits(bb) do
10:      moves.APPEND(Move(sq, dest))
11:   end for
12: end for
13: return moves
14: end function

```

Takeaways and Performance In Phase II, the focus shifted from ease of development to performance. The results of the performance test, after the refactoring of the move generators, were a huge surprise. Even though a lot of the engine was still not migrated to use BitBoards, the speed increased by more than 4 times, from 63.000 NPS to 260.000 NPS, taking it a step closer to the 1.000.000 NPS goal. This huge jump made it clear that the use of bit-wise operations over object-centered operations was the answer to a lot of the performance issues; however, the rest of the migration was put aside until the immutability overhead problem was solved.

Table 3.2: Perf Results for Phase II Engine

Position	Depth	Leaf Nodes	Execution Time (s)
Position 1	5	4,865,609	18
Position 2	4	4,085,603	20
Position 3	6	11,030,083	28
Position 4	5	15,833,292	75
Position 5	4	2,103,487	9
Position 6	4	3,894,594	16

3.4.3 Phase III: Mutable Core

During Phase III, the main focus was to remove the immutability overhead. As found in Phase I, a big performance bottleneck was the need to always create new objects after executing a move. The benefits it offered were the ease of undoing moves and the ease of just calculating everything from scratch, instead of having to pay attention to all side effects that occur after a move is made. Now that performance took priority, these benefits lost their value. The refactoring process was focused around two methods inside the `Board` class, an existing one named `executeMove()`, and a new one named `undoMove()`.

The Refactoring of the `executeMove()` Method This method serves as one of the most important and most utilized methods inside of the engine. It takes as an input a `Move` object and executes it on the board. Before this phase, it would create and return a brand new `Board`, where the given move was executed. This slowed everything down, as every piece of information contained inside the board (legal moves for both sides, piece `BitBoards`, etc.) would need to be recalculated. To fix this, a single instance of the `Board` class was used, and every change was done in place. For example, instead of recalculating all `BitBoards` from scratch, only the `BitBoards` of the moving piece and optionally the captured piece were updated.

The Creation of the `undoMove()` Method The undoing of moves is a very important functionality; it is used inside the correctness and performance testing, as well as in the move search function, which is why it needs to be a fast operation. The implementation was the exact opposite of the `executeMove()`, each step was undone to bring the board to the state it was in before.

Takeaways and Performance After the initial round of refactoring, the results were not as good as expected. There was a speed boost of only about 20%, from 260.000 NPS to 323.000 NPS. The problem was still that there were far too many recalculations. While editing the existing `Board` instance saved some time, a lot of its attributes, like

legal moves for both sides, still needed to be recalculated from scratch when doing and undoing moves. To solve this problem, a `MoveHistoryStack` was added, which stored the state of the board after every move. As a game of chess cannot exceed 50-75 moves, the stack could remain a manageable size, while offering a huge performance boost. When a move was made, the initial information about the board was stored inside the stack, and then the actual board was updated to represent the new move. When a move was undone, the information would be popped from the stack and the board would be updated with that data. This almost doubled the speed of the engine, to about 630.000 NPS.

Table 3.3: PerfT Results for Phase III Engine without Stack

Position	Depth	Leaf Nodes	Execution Time (s)
Position 1	5	4,865,609	14
Position 2	4	4,085,603	16
Position 3	6	11,030,083	23
Position 4	5	15,833,292	60
Position 5	4	2,103,487	7
Position 6	4	3,894,594	13

Table 3.4: PerfT Results for Phase III Engine with Stack

Position	Depth	Leaf Nodes	Execution Time (s)
Position 1	5	4,865,609	7
Position 2	4	4,085,603	8
Position 3	6	11,030,083	12
Position 4	5	15,833,292	33
Position 5	4	2,103,487	4
Position 6	4	3,894,594	6

3.4.4 Phase IV: Full BitBoard Migration

Now that the main flow of the engine was done, this development phase was focused on cleaning up the code and replacing as many possible object operations with bitwise operations.

Move Refactorisation The first change was the replacement of `Move` objects with a single primitive datatype. This would save time when converting `BitBoard` meshes to objects inside the `moveGeneration()` method, and would make the operations with the `MoveHistoryStack` a bit faster. In *Java*, the `int` datatype represents a 32-bit number, meaning the bits could be divided as follows:

- **Bits 0 - 5:** the index of starting the square.

- **Bits 6 - 11:** the index of destination the square.
- **Bits 12 - 14:** the type of the promoted piece (000 = null, 001 = knight, bishop = 010, rook = 011, queen = 100).
- **Bits 15 - 18:** the type of move that was executed (for example "normal move" = 000, "attack move" = 001, etc.).

Along the replacement of the `Move` class, came the replacement of the way moves were stored inside the `Board`. The two lists that stored the legal moves for each side were replaced by two hash maps. The keys of the map were the positions of every piece, and the value was a `BitBoard` with every possible move, also named *attack mask*. This allowed for a faster way to check if a tile was attacked, making check and castling checks faster. Alongside that, it was faster to convert those attack masks to moves when needed, instead of having to create all the moves for both sides after each move.

Castling Rights As said before, an expensive operation that benefited a lot from `BitBoards` was the validation of castling rights. Initially, the engine would iterate through every possible opponent move and check if no move attacked the squares between the rook and king. This was done after every move. Alongside the attack meshes refactoring, four flags that represented each castling move (queen-side and king-side, for both white and black) were added. This allowed the engine to only check the squares of the corresponding flag.

Board Representation Even though every piece had a corresponding `BitBoard`, the representation inside the `Board` class was still a hybrid between `BitBoards` and a list of `Tile`. Most of the functionalities of the `Board` class were working until this point with both in parallel, so completing the migration was a simple cleanup task.

Takeaways and Performance The full migration to pure `BitBoards` and primitive-based representation of the moves managed to deliver a final performance boost: the speed more than doubled, from 630.000 NPS to 1.600.000 NPS, exceeding the goal of 1.000.000 NPS. By eliminating the bulk of object-allocation overhead and unifying all core operations to use bitwise operations, the engine reached a satisfying speed that is more than enough for this thesis.

Table 3.5: Perf Results for Phase IV Engine

Position	Depth	Leaf Nodes	Execution Time (s)
Position 1	5	4,865,609	4
Position 2	4	4,085,603	3
Position 3	6	11,030,083	4
Position 4	5	15,833,292	11
Position 5	4	2,103,487	1
Position 6	4	3,894,594	2

3.5 Final Engine Flow

After walking through the evolution of the engine's design, a clearer, unified description of the step-by-step move generation and execution pipeline is needed.

Initialization : At the start of the game, the board is initialized with the initial board configuration. This means that the move-maker is set to white, the `PiecesBitBoards` is initialized to the starting position, all castling capability flags are set to true, and all legal moves are calculated for each side.

Move Generation : All `BitBoards` within `PiecesBitBoards` are traversed, and for each piece, a move `BitBoard` containing every pseudo-legal move is generated and put inside the hash map. Alongside that, a unified attack mesh containing all attacking possibilities of each side is created.

Move Generation for Move Maker : The move maker's hash map of pseudo legal moves gets converted to legal moves, by checking which moves result in the friendly king getting checked. After that filter, the castling moves get added according to the current castle rights and availability.

Move Choice : There are two ways in which a move can be chosen; the first one is by a player from the web app, and the other option is by the search function. When a player wants to make a move, he needs to select the desired piece from the graphical interface and then choose which legal move to execute. After choosing, an HTTP request with the desired move is made and the `executeMove()` is called. After that, the search function calls the `executeMove()` method directly with the best move it found.

Move Execution : When a move is chosen, either by the user or search function, the `executeMove()` function is called. The current board and move information gets stored inside the `MoveHistoryStack`, the corresponding `BitBoards`, castling rights, en passant position, and pseudo-legal move hash maps are updated.

Undoing a Move : This method can only be executed by the perft tests and search function. It takes the last entry on the `MoveHistoryStack` and updates all board properties with the information taken from the stack, bringing the board to the state it was in before the previous move.

3.6 Frontend and Backend communication

The frontend and backend coordinate the game loop via three REST endpoints. Each response carries the current board state as a FEN string (produced by the backend's `FenService`) so that the Angular UI can render the board.

- `/starting-board-state`: this endpoint is called at the start. It initializes the game inside the backend and returns the current game state. This includes a FEN string representing the board and a flag that is used to mark the winner.
- `/get-moves-for-position`: this endpoint is called when a piece is selected from the user interface. The position of the selected piece is sent to the backend and it returns a list of all possible legal moves for the selected piece.
- `/make-move`: this endpoint is called after a move is selected. The move is sent back to the backend and gets executed. After executing, the engine makes its own move in response and sends back the game state where both moves were executed.

Chapter 4

Search Algorithm

4.1 Search Overview

In this chapter, the move-selection pipeline, that turns a board position into a concrete move choice, will be described. At its core lies a multi-threaded alpha-beta search with Principal Variation Search (PVS), wrapped in iterative deepening. Each root-move search gets run in parallel across multiple CPU cores, coordinated via a shared transposition table. The leaf evaluations call out to a machine-learning model, which returns a float positive for white's advantage, negative for black's.

4.2 Alpha-Beta Search

Algorithm Description The core search is a traditional alpha-beta procedure, enhanced by Principal Variation Search (PVS) and aspiration windows. At each node, two bounds— α (the best guaranteed lower bound) and β (the best guaranteed upper bound)—define a search window. The very first move is examined with the full window $[\alpha, \beta]$ to establish an initial score. All later moves are first probed with a narrow “null-window” search just below the current α ; this quickly identifies any move that can surpass the bound without exploring its entire subtree. If such a probe returns a score inside $[\alpha, \beta]$, a full-window re-search is performed to obtain the precise value. This selective re-search minimizes redundant exploration while ensuring exact results. Recursion continues until the depth limit (depth = 0) is reached, at which point the static evaluation function is called.

Java Implementation The core method `MoveSearch.findBestMoveInternal` initializes timing and atomic counters, generates and orders the root moves, then enters an iterative-deepening loop from depth 1 to `MAX_DEPTH`. At each depth, one task per root move is submitted to a fixed-size thread pool. Each task clones the `Board`, ap-

plies its assigned root move, and invokes `alphaBeta(board, depth-1, -beta, -alpha, ply+1, isPVNode, threadId)`. Upon completion, a `SearchResult`, containing the move, its score, search depth, and principal variation, is recorded in a concurrent transposition table.

The `alphaBeta` routine encompasses:

- Transposition-table lookups and cutoffs via `lookupTranspositionTable()`,
- Move ordering based on the transposition-table move, killer-move entries, and the history heuristic,
- Alpha and beta cutoff detection with atomic updates to `cutoffs`,
- Recursive calls to itself or to the PVS wrapper `searchMove()`,
- Leaf evaluation by `evaluate(board)`, which returns a floating-point advantage from the ML model.

4.3 Move Ordering Heuristics

Effective alpha-beta pruning depends on examining the most promising moves first. The `orderMoves()` routine assigns a composite score to each move according to the following criteria:

1. The move retrieved from the transposition table is given highest priority.
2. Captures are ranked by the Most Valuable Victim-Least Valuable Attacker (MVV-LVA) heuristic.
3. Two “killer” moves per ply—moves that previously caused cutoffs—are promoted next.
4. Remaining non-capture moves are ordered by their history heuristic scores.

By sorting moves in descending score order, early cutoffs are maximized, and the effective branching factor is minimized.

4.4 Transposition Tables and Zobrist Hashing

Each position is encoded as a 64-bit Zobrist key via `computeZobristHash()`. These keys index into a fixed-size array of `TranspositionEntry`, where each slot records the position’s key, search depth, best move, evaluation value, and node type (Exact, Lower, or Upper bound). During search, `lookupTranspositionTable()`

probes the table: if an entry with a matching key and depth $\geq$ the current search depth is found, and its node type and stored bounds allow, the cached evaluation is reused or an early cutoff is triggered. Once a node has been fully evaluated, `storeTranspositionTable()` updates the table using a replacement policy that retains entries for deeper or identical positions.

4.5 Iterative Deepening

Instead of searching directly to the target depth, the engine conducts a series of depth-limited searches at increasing depths (1, 2, ..., `MAX_DEPTH`). Each completed iteration produces a principal variation (PV), which is then used to prioritize move ordering in the next, deeper search. Iterative deepening also ensures that a valid move is always available if the allotted time expires, and the process terminates early upon reaching a mate score or exhausting the time budget.

4.6 Aspiration Windows

Rather than re-searching each depth with the full $(-\infty, +\infty)$ window, the search is first attempted with a narrow “aspiration window” centered on the previous iteration’s score. Concretely, if *bestScore* is the value returned at depth $d - 1$, then at depth d the bounds are set to

$$\alpha = \text{bestScore} - \frac{\text{ASPIRATION_WINDOW}}{100}, \quad \beta = \text{bestScore} + \frac{\text{ASPIRATION_WINDOW}}{100}.$$

By tightening the window, alpha-beta pruning can more quickly detect cutoffs when the true score remains close to *bestScore*, reducing node expansions in the common case where evaluations change only slightly between depths. If the search returns a value below α or above β , indicating that the true score lies outside the aspiration window, the engine immediately retries the same depth with the unbounded window $(-\infty, +\infty)$. Two atomic counters—`aspirationFailLows` and `aspirationFailHighs` track these failures, providing feedback on whether the window size should be adjusted. Overall, aspiration windows strike a balance between pruning effectiveness and the cost of occasional re-searches.

4.7 Concurrency Model

Root-move parallelism is achieved via a fixed-size `ExecutorService` whose thread count matches the number of available CPU cores. For each iterative-deepening

depth, one `Callable` task per root move is submitted to this pool. Internally, each task:

- Clones the current `Board` state and applies its assigned root move.
- Executes a depth-limited alpha-beta search (with PVS) on that board copy.
- Stores its `SearchResult`-including score and principal variation-into a concurrent map.

A concurrent set (`currentRootMoves`) prevents duplicate scheduling of the same move, ensuring that each root move is explored exactly once. Shared structures—such as the transposition table array, killer-move tables, and history heuristic matrix—are protected by atomic operations or brief synchronized blocks, minimizing contention. After submission, the main thread waits on the array of `Future` objects with a flexible timeout (via `waitForFuturesCompletion`), which allows tasks to complete within the remaining time budget while also handling thread interruptions and timeouts. Upon completion or timeout, the executor is shut down using `shutdownNow()` and `awaitTermination()`, ensuring that no stray threads remain.

4.8 Search Statistics

A set of atomic counters is maintained throughout each search to monitor performance and guide tuning:

- `nodesSearched`: total number of nodes visited by the alpha-beta routine;
- `evaluations`: number of times the ML evaluation function is invoked;
- `ttHits`: count of successful transposition-table lookups;
- `cutoffs`: number of alpha or beta cutoffs performed;
- `aspirationFailLows/aspirationFailHighs`: tally of aspiration-window failures that triggered a full-window re-search.

At the end of each depth, these metrics are logged via `logSearchStatistics()` and provide immediate feedback on pruning effectiveness, cache utilization, and overall search behavior.

In the following table, the statistics of a 5-second limit search are displayed. We can see that the search function ran for only 4 seconds; this means that it saw the time limit approaching and decided to abort the search and keep the best move until

then. Within those 4 seconds, it reached a depth of 5 and checked 34,641 nodes. From those, 28,862 needed to be evaluated by the evaluation function, 1,656 could get their values from the transposition table, 3,632 did not need evaluations because they were cut off and the rest of 491 were not evaluated due to early termination.

Metric	Value
Search depth	5
Elapsed time	4002 ms
Nodes searched	34,641
Evaluations	28,862
Transposition-table hits (<code>ttHits</code>)	1,656
Cutoffs	3,632

Table 4.1: Sample Search Statistics

Even though the engine can generate over a million positions per second, the search is extremely limited by the evaluation function and the time it takes the model to run. For the statistics above, the final model was used. The inference time would create a big problem as it limited the size of the model used. A very precise model would require more time to think, but that would limit the search depth. A small model would be able to look a lot deeper, but its predictions would be less accurate. To find a balance, we decided to use a model that would be able to reach a depth of 4 in 5 seconds. The final model was just able to start evaluating nodes on the 5th layer, while remaining under 5 seconds.

4.9 Evaluation Function

Leaf nodes are handled by a single evaluation call:

```
float evaluate(Board board);
```

This method returns a signed floating-point score—positive values indicate an advantage for White, negative for Black. The architecture, training process, and integration details of this learned evaluator are going to be the focus of the next chapter.

Chapter 5

Machine Learning Models

5.1 Introduction

In traditional chess engines, position evaluation relies heavily on handcrafted evaluation functions: material counts, piece-square tables, pawn-structure heuristics, king safety metrics, and numerous hand-tuned coefficients. Crafting and tuning these evaluation terms requires deep domain knowledge, extensive trial-and-error, and often results in systems that must be adjusted whenever opening theory or positional understanding evolves.

The primary goal of this chapter is to build a neural-network-based evaluator that replaces these handcrafted rules and heuristics. Instead of manually specifying every factor, the model will *learn* to distinguish good and bad positions by training on a large database of positions labeled with high-quality evaluations from an existing engine (Stockfish). By doing so, it should capture both tactical and strategic considerations automatically, discovering which patterns correlate with advantage or disadvantage, without explicit human encoding of each positional theme.

5.2 TensorFlow Framework

All of the neural-network models in this work were implemented in TensorFlow using its high-level Keras API. TensorFlow was chosen because it provides:

- **Flexible model definitions.** Simple feed-forward prototypes were built with `Sequential()`, while the final CNN and ResNet+Attention architectures leveraged the Functional API (`Input()`, `Model(...)`) to cleanly express multi-input and residual/attention branches.
- **Efficient data pipelines.** The `tf.data` module and TFRecord format enabled streaming large position-evaluation datasets directly from disk with

`TFRecordDataset`, `map / batch / prefetch` for maximum I/O throughput.

- **Built-in training utilities.** Standard callbacks (`ReduceLROnPlateau`, `EarlyStopping`, `ModelCheckpoint`, `TerminateOnNaN`) and optimizers (`Adam`) were used out-of-the-box to manage learning-rate schedules, overfitting prevention, and training stability.
- **Custom losses and mixed precision.** The custom weighting functions (e.g. zero-focused WMSE and Huber) were defined easily via TensorFlow's loss-function API, and mixed-precision float16/float32 training was enabled through `tf.keras.mixed_precision` for a substantial speedup.

This combination of high-level modeling, efficient data ingestion, and hardware-friendly training made TensorFlow the natural choice for rapidly iterating over multiple architectures.

5.3 Data Pipeline

5.3.1 Data Generation from PGN

The raw training data comes in the form of *PGN-files*, from the public *Lichess.com* database. A custom Python script filtered these games to include only those played by humans with an *Elo* rating above 2000, and for which a *Stockfish* evaluation was already available. From each selected game, forty positions were sampled at random, and for each position, the following pair was stored in a relational *PostgreSQL* database:

- `FEN`: the FEN string describing the board state.
- `evaluation`: the Stockfish score (in centipawns) for the side to move.

Over the course of model development, the dataset grew from roughly 200,000 positions to over 5 million.

5.3.2 Data Visualization and Details

The final data set had a total of 5,233,865 unique positions, evaluated from -1 to 1 centipawns. The positions whose evaluations were outside this interval were scaled back to the ± 1 limits. This was done in order to avoid extreme values that could affect training. About 1,7 million positions are in the opening phase of the game, 3,2 million are in the middle game, and 300 thousand are in the endgame. This

distribution assured that more data was used for more common positions. The bulk of the data was around the value 0, because during the game of chess, the positions have a pretty equal advantage for both players. This proved to be a challenge when training the model, as the values around 0 needed to be more precise, while the ones around the limits did not need huge accuracy.

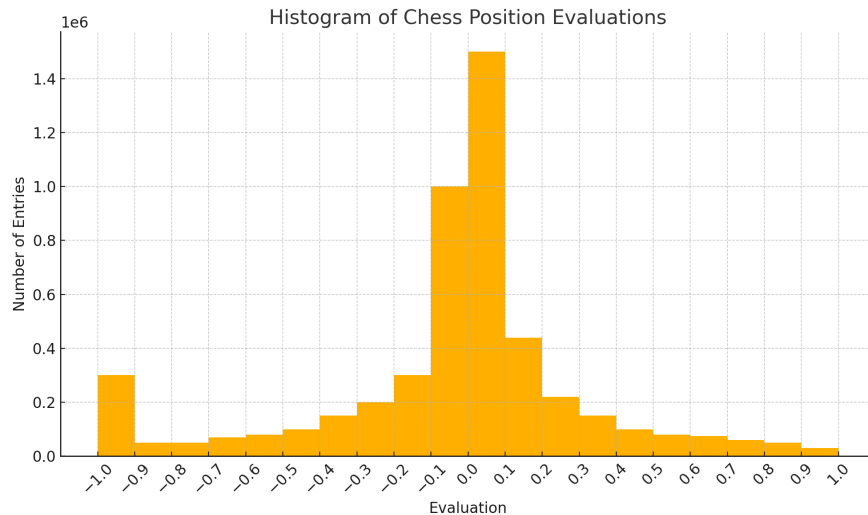


Figure 5.1: The number of positions with similar evaluation scores.

5.3.3 TFRecord Serialization

Early experiments loaded data directly from the database into *NumPy* arrays, but as the volume increased, this approach became I/O bound and memory limited, so another input format needed to be used. TensorFlow’s TFRecord format provided a compact, binary container for sequences of examples that could be read and pre-processed on the fly, without ever materializing the whole dataset in RAM. In our pipeline, each example packed three fields:

1. `board_tensor`: A serialized `float32` tensor of shape `[8, 8, 12]`, representing an encoded representation of the pieces positions.
2. `extra_features`: A serialized `float32` vector of length 6, encoding side-to-move, four castling-rights flags, and the en-passant flag.
3. `evaluation`: A `float32` scalar giving the Stockfish score (in centipawns).

Writing TFRecords The files are generated by a Python script that fetches batches of pairs of `(fen, evaluation)` from the SQL database, then converts each FEN into a `[8, 8, 12]` bitboard tensor and the 6-element extra-feature vector. After conversion and serialization, these are appended to a file. Multiple such files are used in order to increase the efficiency of data processing during model training.

5.3.4 Feature Encoding: BitBoards vs. One-Hot

Two distinct board representations were explored:

- *One-Hot Encoding* Each of the 64 squares is represented by a 12-dimensional one-hot vector (6 piece types $\times$ 2 colors), yielding an $8 \times 8 \times 12$ tensor with exactly one "1" and eleven "0" values in each piece-type channel per square.
- *BitBoard Encoding* A more compact $8 \times 8 \times 12$ tensor of 0/1 values, where each channel is stored internally as a 64-bit integer mask. This matches the engine's native BitBoard representation and avoids costly conversion steps.

In practice, both encoding styles yielded nearly identical validation errors, so the final pipeline utilized BitBoards to eliminate redundant preprocessing in the backend.

5.4 Convolutional Neural Network Architecture

5.4.1 Initial Baseline (Version 1)

This is the first model that was developed for this thesis. It was trained on 200K positions and worked by taking a position, creating a single vector from it, and then running it through some dense and dropout layers, and, in the end, giving a single output, the predicted evaluation of the position.

Architecture:

Flatten() layer: flattens the $8 \times 8 \times 12$ bitboard input into a 768-vector.

Dense(512, activation=ReLU, kernel regularizer= $\ell_2(10^{-3})$).

Dropout(0.2).

Dense(128, activation=ReLU, kernel regularizer= $\ell_2(10^{-3})$).

Dropout(0.2).

Dense(64, activation=ReLU).

Dense(1, activation=linear): scalar evaluation.

Data Preparation & Training Regime The training process involves splitting the data into 80% for training (`x_train`) and 20% for validation (`x_val`). The training process uses the *Adam* optimizer with a learning rate of 0.001 and employs *Mean Squared Error* (WMSE) as the loss function. The process also incorporates callbacks: *ReduceLROnPlateau* with a factor of 0.5, patience of 5 epochs, and a minimum learning rate of 0.000001, alongside *EarlyStopping* with a patience of 10 epochs and an option to restore the best weights. The training is set to run for up to 50 epochs but typically concludes around 40 to 45 epochs.



Figure 5.2: Training vs. validation WMSE for CNN model version 1.

Behavior & Observations The training loss falls rapidly from > 1.0 to ≈ 0.8 within five epochs. Validation loss shows bumps around epochs 5-15 (tactical noise), and occasional spikes after epoch 20 (overfitting). Learning-rate reductions produce stepwise improvements. Final WMSEs settle near 0.55 (train) and 0.68 (val). The flattening destroys spatial context, which means it cannot learn local tactics. With approximately 500k parameters, the model is prone to overfitting on rare tactical outliers.

5.4.2 Hybrid CNN + Dense Extra-Features (Version 8)

The following model is the eighth version of the CNN architecture. Compared to the first version, which was only trained with about 200k positions, this one was trained with about 3 million. In addition to the larger dataset, an additional input vector with more board information was used. The board input was not flattened directly; it

was first run through 3 *Conv2D* layers, which were used to help the model find local patterns and structures. Only after these layers and the concatenation of the extra features would the model use *Dense* layers.

Architecture:

Board Input ($8 \times 8 \times 12$)

Conv2D(64, 5×5, activation=swish, padding=same) → BatchNorm → swish

Conv2D(128, 3×3, activation=swish, padding=same) → BatchNorm → swish

Conv2D(256, 3×3, activation=swish, padding=same) → BatchNorm → swish

Flatten()

Extra-Features (6-vector): Dense(128, activation=swish).

Concatenate([board_features, extra_features]).

Dense(256, activation=swish, kernel_reg= $\ell_2(5 \times 10^{-3})$) → Dropout(0.25).

Dense(64, activation=swish, kernel_reg= $\ell_2(5 \times 10^{-3})$) → Dropout(0.35).

Dense(1, activation=linear).

Training Regime This model was compiled with Adam($\eta = 3 \times 10^{-4}$), and WMSE was used to monitor its performance. The same callback functions were used as the baseline model. The training limit was 50 epochs.

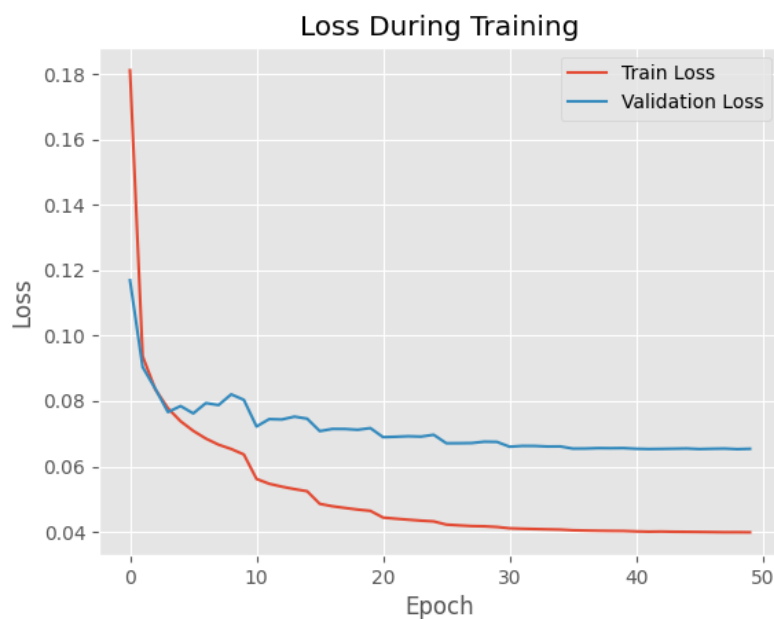


Figure 5.3: Training vs. validation WMSE for CNN model version 8.

Behavior & Observations The loss curves display three distinct phases. In the first five epochs, both training and validation losses plummet, training WMSE falls from about 0.18 to 0.065 and validation from roughly 0.12 to 0.075, showing that the convolutional backbone quickly captures dominant patterns. From epochs 5-15, the validation curve plateaus and even oscillates slightly (peaking around 0.08), indicating that the model is beginning to overfit despite L2 and dropout. Each time `ReduceLROnPlateau` halves the learning rate (visible as small inflection points around epochs 10 and 20), both curves dip again, suggesting that lower step sizes let the network fine-tune more effectively. After epoch 20, losses converge more smoothly: training loss steadily decays to about 0.04 by epoch 50, while validation settles near 0.065, yielding a stable val-train gap of 0.025 that reflects both strong fit and good generalization.

5.4.3 Final Convolutional Architecture (Version 9)

This final model improves on Version 8 by stacking several small 3×3 filters instead of one big 5×5 , which lets it pick up finer board details with fewer weights; it replaces the flattening step with global average pooling to cut hundreds of thousands of connections and focus on each feature map's overall strength rather than exact location; it shrinks the extra-features branch to just 32 neurons so most capacity goes toward spatial patterns; and after merging these streams it uses three dropout-and-L2-regularized dense layers before a tanh output that keeps evaluations between -1 and 1. Trained on 5 million positions, this leaner, deeper design captures rich tactics while generalizing more reliably to new games.

Architecture:

Conv2D(64, 3×3 , padding=same, kernel_reg= $\ell_2(10^{-4})$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU

Conv2D(64, 3×3 , padding=same, kernel_reg= $\ell_2(10^{-4})$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU

Conv2D(128, 3×3 , padding=same, kernel_reg= $\ell_2(10^{-4})$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU

Conv2D(128, 3×3 , padding=same, kernel_reg= $\ell_2(10^{-4})$) $\rightarrow$ BatchNorm $\rightarrow$ ReLU

GlobalAveragePooling2D

Extra-Features (6-vector): Dense(32, activation=ReLU).

Concatenate([board_features, extra_features]).

Dense(128, activation=ReLU, kernel_reg= $\ell_2(10^{-4})$) $\rightarrow$ Dropout(0.3).

Dense(64, activation=ReLU, kernel_reg= $\ell_2(10^{-4})$) $\rightarrow$ Dropout(0.3).

Dense(32, activation=ReLU, kernel_reg= $\ell_2(10^{-4})$) → Dropout(0.3).

Dense(1, activation=tanh).

Training Regime This model was compiled with Adam($\eta = 3 \times 10^{-4}$), and WMSE was used to monitor its performance. The same callback functions were used as the baseline model. The training limit was 40 epochs, but it would.

Behavior & Observations The loss curves are again split into three phases. In the first five epochs, both curves plummet, training WMSE drops from about 0.12 to 0.08 and validation from roughly 0.12 to 0.075, showing the network quickly grasps core spatial relationships. From epochs 5-15, validation plateaus and oscillates between 0.075 and 0.085, signaling that the model is starting to overfit despite L2 and dropout. Small inflection points around epochs 10 and 20 mark each *ReduceLROnPlateau* halving of the learning rate, after which both losses dip again, although a pronounced spike at epoch 20 in the validation curve highlights remaining sensitivity to outliers. After epoch 20, training loss steadily declines to about 0.04 by epoch 40, while validation settles near 0.065, maintaining a stable val-train gap of roughly 0.025 that reflects robust learning and good generalization.

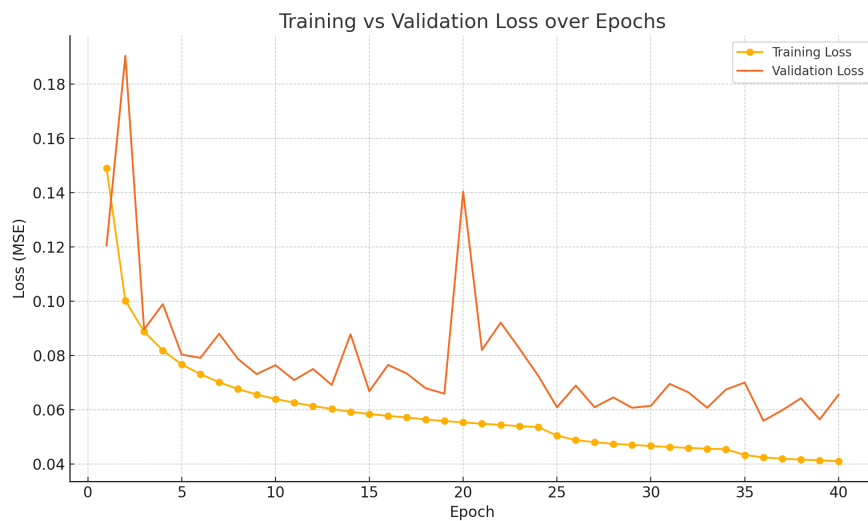


Figure 5.4: Training vs. validation WMSE for CNN model version 9.

Advantages Version 9 compared to Version 8 Even though version 9 has a lot more noise, we chose to use this model as its prediction speed was better than version 8. Despite its smooth decline, Version 8's near-monotonic loss curve actually raises concerns about under-utilized capacity and over-regularization. The almost bump-free descent hints that the network is playing it too safe, never venturing beyond a shallow local minimum.

5.5 Weighted MSE vs. Huber Loss Functions

All networks were trained with a custom *weighted mean-squared error* (WMSE) loss that emphasizes positions whose true evaluation lies near zero. Concretely, if y_i is the ground-truth Stockfish score and $\hat{y}_i$ the model prediction, then

$$L_{\text{WMSE}} = \frac{1}{N} \sum_{i=1}^N w(y_i) (y_i - \hat{y}_i)^2, \quad w(y) = 1 + 5 \frac{\max(0, \delta - |y|)}{\delta},$$

with $\delta = 0.15$. This “triangular” weight peaks at $y = 0$ (balanced positions) and tapers linearly to 1.0 at $\pm\delta$.

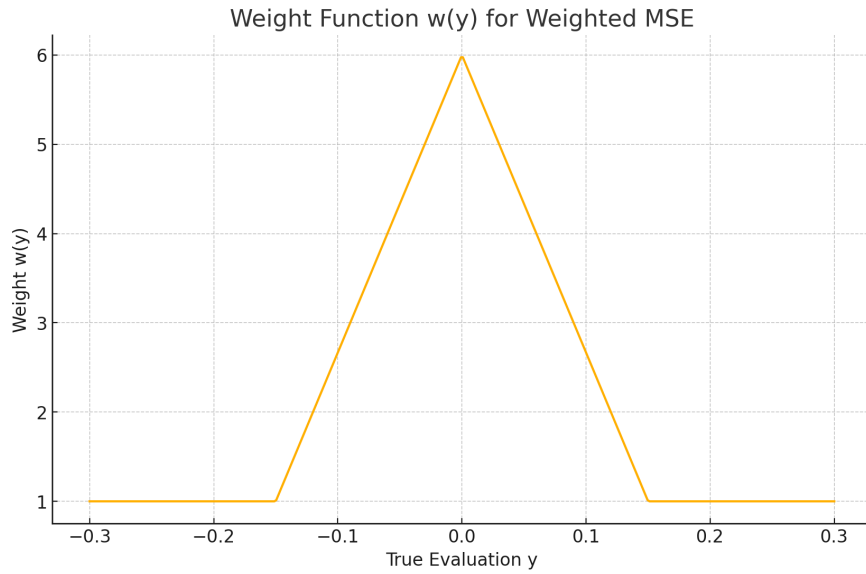


Figure 5.5: Representation of the WMSE.

A *zero-focused Huber* variant was also tested, combining the standard Huber loss with the same proximity weight $w(y)$. Although more robust to large outliers, the Huber-based objective produced larger and more erratic validation loss and ultimately underperformed compared to the WMSE. Accordingly, the weighted MSE loss was adopted for all subsequent experiments.

$$\ell_{\text{Huber}}(e) = \begin{cases} \frac{1}{2} e^2, & |e| \leq \delta, \\ \delta(|e| - \frac{1}{2} \delta), & |e| > \delta, \end{cases} \quad e = y - \hat{y}$$

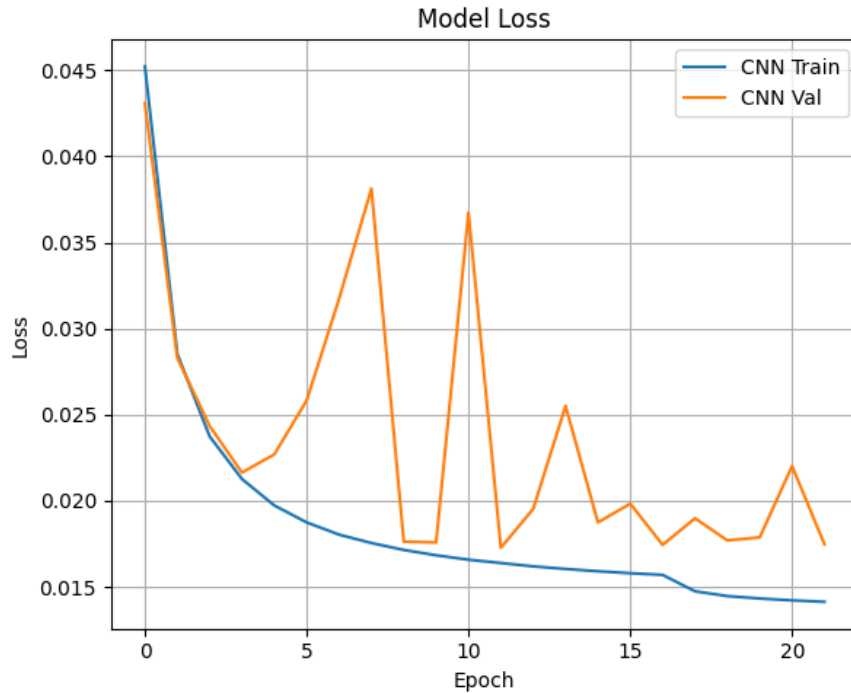


Figure 5.6: Training vs. validation Huber Loss for CNN model version 9.

5.6 Training Infrastructure

All initial model-training experiments were carried out on a local desktop equipped with an NVIDIA RTX 3060 GPU (8 GB VRAM) alongside an Intel i5-12400F CPU and 16 GB of system RAM. For the simplest, fully connected architectures and smaller subsets of the data, this setup sufficed and training jobs completed in a reasonable amount of time. However, as both the dataset size and model complexity increased, training times ballooned to around 14 hours per model. At that point, the local GPU’s memory constraints and I/O bandwidth became the dominant bottlenecks.

To overcome these limitations, all subsequent large-scale training was migrated to Google Colab. TFRecord files were synced to Google Drive and streamed directly to avoid repeated database access. Colab’s A100 GPU instance (40 GB VRAM) delivered both a higher compute throughput and substantially larger on-device memory, enabling much larger batch sizes. Under this configuration, end-to-end training for even the most complex model completed in roughly 1-2 hours, a 7-14 $\times$ speedup compared to the original local runs. I/O performance was similarly improved by prefetching and sharding the TFRecord streams across multiple files, ensuring the GPU remained fully utilized throughout each epoch.

5.7 In-Game Performance

The current model displays a decent performance when taking into account the resource limitations. The strong point is definitely the performance while evaluating positions where the advantage of one player is a bit more clear. As said during the analysis of the data set, its weak point is the accuracy when working with positions of about equal score. While not performing horribly, the accuracy is also affected by the limited thinking time, caused by the search function's time limitation and the long inference time caused by the hardware it runs on.

Chapter 6

Conclusion

6.1 Summary of Contributions

This work has demonstrated that a single developer, armed with foundational chess knowledge and modern software tools, can build a fully functional chess engine from the ground up. Key achievements include:

- A *standalone engine* featuring FEN parsing, perft-validated move generation, and a multi-threaded alpha-beta search loop augmented by move ordering, transposition tables, and iterative deepening.
- A *systematic study* of classical optimizations: magic bitboards, mutable core with history stacks, custom primitive move encoding, showing a phase-by-phase speedup from 63k NPS to 1.6M NPS.
- A *neural network evaluator* trained on millions of Stockfish-annotated positions, consuming only FEN-derived bitboard inputs, and delivering continuous evaluation scores in place of handcrafted heuristics.
- A *hybrid integration* in which the learned evaluator seamlessly plugs into the search loop, enabling a fully automated engine pipeline from move generation through position evaluation.

6.2 Answers to Research Questions

1. **Effect of engine optimizations:** Phased refactorings, from object-heavy to BitBoard centric data structures and from immutable to mutable board state, produced dramatic throughputs increases ($63 \rightarrow 260 \text{ K} \rightarrow 630\text{k} \rightarrow 1.6\text{M NPS}$), validating the impact of each technique.

2. **Impact on the search tree:** Enhanced move ordering, transposition-table lookups, and aspiration windows sharply reduced effective branching factors and increased cutoff rates, allowing deeper searches within fixed time budgets.
3. **Viability of a learned evaluator:** A model trained purely on existing engine evaluations achieved strong correlation (validation MSE ≈ 0.05 - 0.08) with Stockfish scores, demonstrating that supervised regression on FEN inputs can serve as a competent evaluation function.
4. **Trade-offs of model complexity vs. inference cost:** A compact CNN offered fast inference with modest accuracy, while a deeper model slightly improved evaluation quality at the cost of slower per-node evaluation. In practice a balance between speed and accuracy is the best.

6.3 Limitations

- **Absence of advanced pruning:** Techniques such as quiescence search, null-move pruning, and late-move reductions were not integrated and could yield further depth gains.
- **No endgame or opening knowledge:** Without tablebases or opening books, the engine relies entirely on search and learned evaluation, limiting its play in extreme endgames or established opening theory.
- **Data and model scope:** Training was confined to Stockfish-annotated master games with ELO ≥ 2000 . Self-play or reinforcement learning was not explored, and the evaluator has not been fine-tuned on unusual positions.

6.4 Future Work

Several ways remain for improving both the engine and the evaluator:

- *Integrate advanced search prunings* (quiescence, null-move, late-move reductions) to boost effective depth and stability.
- *Incorporate endgame tablebases and opening libraries* to cover phases where raw search or learned evaluation may falter.
- *Explore reinforcement learning* or self-play to refine the evaluator beyond direct imitation of Stockfish, potentially capturing strategic concepts not present in the training data.

- *Optimize inference* via quantization or model distillation, reducing per-node evaluation cost and permitting deeper searches under strict time controls.
- *Extend the user interface* with graphical board analysis tools, move lists, and adjustable engine strength settings.

By building on the foundation laid here, combining classical optimizations with modern machine learning, the next generation of the engine can approach even more ambitious goals in automated chess play and research.

Bibliography

- [Alp21] Ethem Alpaydin. *Machine learning*. MIT press, 2021.
- [AVAB⁺07] G Adelson-Velskii, V Arlazarov, A Bitman, A Zhivotovskii, and A Uskov. Programming a computer to play chess. *Russian Mathematical Surveys*, 25:221, 10 2007.
- [Ban17] Ankan Banerjee. Perft(15), 2017.
- [Bre96] Gerd Brewka. Artificial intelligence—a modern approach by stuart russell and peter norvig, prentice hall. series in artificial intelligence, englewood cliffs, nj. *The Knowledge Engineering Review*, 11(1):78–79, 1996.
- [Bri00] James Bright. Crafty – command-driven chess engine documentation. Technical report, University of Texas at Dallas, 2000.
- [BU02] Dennis M Breuker and Jos WHM Uiterwijk. Transposition tables in computer chess. *Department of Computer Science, University of Limburg*, 2002.
- [Che25] Chess Programming Wiki. Perft_Results. https://www.chessprogramming.org/Perft_Results, 2025. Accessed: 2025-05-26.
- [Com78] Computerworld Staff. Chess-analysis software rsce-1 evaluated by uscf. *Computerworld*, pages 12–15, Apr 1978.
- [dEF21] Federation Internationale des Echecs (FIDE). Laws of chess, 2021.
- [FL81] Aviezri S Fraenkel and David Lichtenstein. Computing a perfect strategy for $n \times n$ chess requires time exponential in n . In *International Colloquium on Automata, Languages, and Programming*, pages 278–293. Springer, 1981.
- [GG21] Kaan Gokcesu and Hakan Gokcesu. Generalized huber loss for robust learning and its efficient minimization for a robust statistics. *arXiv preprint arXiv:2108.12627*, 2021.

- [HS24] Pål Hammersborg and Ingrid Strümke. Information based explanation methods for deep learning agents—with applications on large open-source chess models. *Scientific Reports*, 14:20174, August 2024. Received: 1 February 2024; Accepted: 20 August 2024; Published: 30 August 2024.
- [Kan07] Pradyumna Kannan. Magic move-bitboard generation in computer chess. *Preprint*, 2007.
- [KM75] Donald E Knuth and Ronald W Moore. An analysis of alpha-beta pruning. *Artificial intelligence*, 6(4):293–326, 1975.
- [Kor85] Richard E Korf. Depth-first iterative-deepening: An optimal admissible tree search. *Artificial intelligence*, 27(1):97–109, 1985.
- [lik] like a wizard. Review of different board representations in computer chess. <https://shorturl.at/nD34P>.
- [Men22] Patricio Mendoza. Alpha-beta pruning algorithm: The intelligence behind strategy games. 05 2022.
- [New12] Monty Newborn. *Kasparov versus Deep Blue: Computer chess comes of age*. Springer Science & Business Media, 2012.
- [ON15] Keiron O’shea and Ryan Nash. An introduction to convolutional neural networks. *arXiv preprint arXiv:1511.08458*, 2015.
- [PM10] David L Poole and Alan K Mackworth. *Artificial Intelligence: foundations of computational agents*. Cambridge University Press, 2010.
- [Sha50] Claude E Shannon. Xxii. programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950.
- [SHS⁺18] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- [Smi78] R. C. Smith. RSCE-1: A cobol chess analysis program. Submitted to USCF; reported in Computerworld, 1978.
- [Spe21] Logan Spears. Train your own chess ai, August 2021. [Online; accessed 26-May-2025].

- [Vid20] Analytics Vidhya. A comprehensive guide to loss functions: Part 1 — regression, 2020. [Online; accessed May 27, 2025].
- [VSP⁺17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [VWB16] Andreas Veit, Michael J Wilber, and Serge Belongie. Residual networks behave like ensembles of relatively shallow networks. *Advances in neural information processing systems*, 29, 2016.
- [Zob90] Albert L Zobrist. A new hashing method with application for game playing. *ICGA Journal*, 13(2):69–73, 1990.
- [ZY20] Hongming Zhang and Tianyang Yu. Alphazero. *Deep Reinforcement Learning: Fundamentals, Research and Applications*, pages 391–415, 2020.